IMPLEMENTING POST-QUANTUM SECURE EXOTIC SIGNATURE SCHEMES IN BLOCKCHAINS

A dissertation submitted to The University of Manchester
for the degree of Master of Science
in the Faculty of Science and Engineering

2026

Royce Steven

Department of Computer Science

Contents

Word Count: 9001

List of Tables

List of Figures

Abstract

Implementing Post-Quantum Secure
Exotic Signature Schemes in Blockchains
Royce Steven
A dissertation submitted to The University of Manchester
for the degree of Master of Science, 2026

Blockchains authenticate transactions with elliptic-curve signatures, which a large quantum computer would break. Basic post-quantum signatures are now standardised, but the *exotic* signatures that give blockchains their advanced features — in particular *adaptor signatures*, the cryptographic core of atomic swaps and payment channels — remain largely paper-only in the post-quantum setting. This dissertation implements and evaluates one such scheme, LAS, a lattice-based adaptor signature, by reusing the CRYSTALS-Dilithium reference primitives, and demonstrates it end-to-end in a post-quantum atomic swap.

The artefact is built *additively*: the entire lattice core is reused unchanged, with no upstream function modified, while the adaptor layer (PreSign, PreVerify, Adapt, Extract), a bit-packed wire encoding with a validating decoder, and the application are added as new code. It passes functional, serialization, tamper and known-answer tests, and an independent second implementation in Rust reproduces the wire format and the pinned digest byte-for-byte under a different compiler and statistical harness.

The primary comparison is LAS against its own simplified Dilithium-style base at identical parameters and primitives, which isolates the adaptor layer's cost; a classical elliptic-curve (ECDSA) adaptor provides a functionality-matched "price of post-quantum" baseline. At the core measurement boundary the adaptor layer

is almost free in computation: PreSign, PreVerify and Adapt each add at most +8.3% over the basic operation they mirror, and Extract is the cheapest of all. The real cost is *communication* — the signature is 99.3% lattice response vector, and an adaptor swap additionally transmits a public-key-sized statement — and that cost reappears as computation once the byte boundary is included, raising the premium to +20.7–90.0%.

Building the same construction on unmodified NIST ML-DSA tests the simplification the implementation rests on, and corrects it: the signing side must indeed be redesigned, but the standard verifier need not be, and it accepts the adapted signature unchanged. That route halves the signature at equal computation while leaving the statement untouched, so it relocates the communication bottleneck rather than removing it.

Taking the protocol to a UTXO ledger, the venue the original construction assumes, sharpens the conclusion: at the application level the signature is not the dominant cost at all. Measured across three configurations, the zero-knowledge proof of knowledge consumes over 98% of a swap, and substituting a lattice prover for a pairing-based one makes the swap *faster* while making it larger. A working, tested, end-to-end post-quantum exotic signature is therefore achievable by reuse of standardised primitives, and the proof system, not the signature, is where a post-quantum swap should next be optimised.

Declaration

No portion of the work referred to in this dissertation has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright

Statement of qualifications and research

[TODO: Only you can write this — state your prior degree(s)/qualifications and any relevant research experience, e.g. "The author holds a BSc in ... from .... No part of the work reported here has been undertaken as part of any previous qualification."]

Acknowledgements

[TODO: Optional but conventional — e.g. thanks to your supervisor Dr Zhipeng Wang, and to anyone else you wish to credit.]

Chapter 1

Introduction

1.1 Context and motivation

Public blockchains authorise every transaction with a digital signature, almost always the Elliptic Curve Digital Signature Algorithm (ECDSA), Schnorr, or Boneh–Lynn–Shacham (BLS) signatures over an elliptic curve. The security of all three rests on the hardness of the elliptic-curve discrete logarithm problem. Shor's algorithm solves that problem in polynomial time on a sufficiently large quantum computer [22], so a future quantum adversary could forge signatures and spend other users' funds. "Post-quantum" (PQ) cryptography replaces these assumptions with problems believed hard even for quantum computers, such as those over lattices.

The U.S. National Institute of Standards and Technology (NIST) has standardised *basic* PQ signatures, notably ML-DSA (the Module-Lattice-Based Digital Signature Algorithm), derived from CRYSTALS-Dilithium [18, 8]. A basic signature authenticates a message and nothing more. Blockchains, however, depend on *exotic* signatures that add functionality on top of a basic scheme: multi-signatures, aggregate signatures, threshold signatures, and *adaptor signatures*. These exotic schemes are what give blockchains scalable consensus, compact multi-party authorisation, and advanced payment protocols. A recent survey documents that, while basic PQ signatures are maturing, their exotic counterparts largely lack efficient, practical implementations in a blockchain setting [5]. Closing that gap — turning paper designs into tested, measured, blockchain-facing artefacts — is the problem this project addresses.

This dissertation focuses on the *adaptor signature*, which ties the act of publishing a valid signature to the simultaneous revelation of a secret witness [19, 1]. That single primitive underpins "scriptless" atomic swaps and payment-channel networks: it lets two parties exchange assets, even across different chains, so that either both transfers happen or neither does, without trusted intermediaries or heavy on-chain scripts. Classically these constructions are mature and deployed; post-quantum they are mostly paper-only, making the adaptor signature a representative, high-value exotic scheme.

Cross-chain exchange first relied on custodial intermediaries; hash-time-locked contracts removed the custodian by locking both transfers under one hash preimage, so claiming one leg reveals the preimage unlocking the other. Such contracts, however, need explicit on-chain script support, place a common hash on both chains — linking the legs — and enlarge the transactions. The adaptor signature answers all three [19, 1] by moving the lock *into the signature itself*: settlement transactions look like ordinary single-signer payments, atomicity is retained, and the witness reaches only the counterparty holding the matching pre-signature. The link removed is the explicit protocol one; timing and amounts can still correlate the legs.

1.2 Subject area and related work

Exotic signatures for post-quantum blockchains. The motivating survey [5] catalogues exotic signature families and identifies the shortage of efficient PQ-secure blockchain implementations as an open challenge.

Lattice signatures and Dilithium. CRYSTALS-Dilithium is a lattice signature in the "Fiat–Shamir with aborts" family [8, 16], its security reducing to the Module Learning With Errors and Module Short Integer Solution problems. It samples a masked commitment, derives a challenge by hashing, computes a response, and *rejects and retries* when the response would leak the secret — the rejection-sampling step characterising the family. Its reference implementation provides the polynomial arithmetic, number-theoretic transform (NTT) and SHAKE-based sampling this project reuses.

Adaptor signatures. Introduced informally as "scriptless scripts" [19] and later formalised [1], the abstraction adds four algorithms to a base signature — PreSign, PreVerify, Adapt, Extract — relative to a hard relation (Y, y): a pre-signature can be *adapted* into a full signature by anyone knowing the witness y, and publishing that signature lets anyone holding the pre-signature *extract* y. Anonymous multi-hop locks generalise this to payment paths [17].

Post-quantum adaptor signatures. Esgin, Ersoy and Erkin proposed LAS, a lattice-based adaptor signature on a Dilithium-style base [9]; it is the design implemented here. Their work is primarily theoretical — definitions, a construction and security proofs — leaving the practical, benchmarked, blockchain-facing implementation open. poqeth studies how PQ primitives are integrated and costed on Ethereum [12], providing a template for the on-chain side.

Classical baseline. The natural answer to "what does post-quantum cost" is a comparison with a *classical adaptor* signature, not merely plain ECDSA. Mature ECDSA-adaptor implementations exist [4]; this project benchmarks against one, stating security parameters explicitly so the comparison is fair (chapter 2).

1.3 Project category, aim, and objectives

Category. This is a *systems implementation and evaluation* project grounded in existing research, not a proposal of a new cryptographic protocol: its emphasis is practical implementation and benchmarking.

Aim. To implement and evaluate a post-quantum exotic signature scheme — a lattice-based adaptor signature reusing the CRYSTALS-Dilithium primitives — and to demonstrate it in a post-quantum blockchain atomic-swap application.

Objectives.

1. Review post-quantum and exotic-signature literature sufficiently to justify selecting the adaptor signature LAS and the Dilithium primitives.

2. Implement LAS additively on the Dilithium reference, reusing the lattice core unchanged and adding the adaptor operations and a byte-level wire encoding.

3. Validate correctness and robustness through functional, serialization, tamper, and known-answer tests, including cross-validation by a second, independent implementation checked against the same known-answer digest.

4. Benchmark LAS fairly — at explicitly stated parameters, with repeated runs and reported variance — against a simplified Dilithium baseline at identical parameters and a classical ECDSA-adaptor baseline, separating computation from communication cost across the five standard signature metrics.

5. Demonstrate an end-to-end post-quantum atomic swap using the implementation — following the protocol of [9] verbatim, including its zero-knowledge proof of knowledge π — and discuss its feasibility.

1.4 Contributions

This dissertation contributes a tested, independently cross-validated implementation of a post-quantum adaptor signature, and measurements of its cost. Four findings carry it. Adaptor functionality proves close to free in computation but expensive in communication: its price is bytes, not time (sections 3.2 and 3.4). The design assertion the implementation rests on — that an adaptor needs an exact verification identity — was tested, not argued, and proved half wrong: PreSign and PreVerify must indeed be new algorithms, yet an unmodified FIPS 204 verifier accepts the adapted signature (section 3.5). At application scale the bottleneck is not the signature but the zero-knowledge proof, leaving the fully post-quantum swap the faster of the two such configurations and the only one Shor does not break (section 3.7). And native on-chain verification, first measured far above the transaction gas cap, fits under it once the same predicate is re-expressed — at the single parameter set and message length measured — while a settled swap sits well inside a UTXO chain's weight limit (sections 3.6 and 3.8). All five objectives were met (table 4.1), subject to two limits throughout: no concrete security is formally analysed, and both applications run over modelled, not live, ledgers.

1.5 Dissertation structure

Chapter 2 describes the methods; chapter 3 reports the measurements and discusses their validity; chapter 4 judges the work against the objectives; chapter 5 concludes, reflects critically, and proposes future work.

Chapter 2

Methodology

2.1 The LAS construction

LAS is a lattice adaptor signature whose base is a simplified, Dilithium-style Fiat–Shamir-with-aborts signature [9, 8]. All objects live in $R_q = \mathbb{Z}_q[X]/(X^d + 1)$, the public matrix has the form $A = [\, I_n \mid A' \,] \in R_q^{n \times (n+\ell)}$, and a key pair is a short secret r with its image $t = A \cdot r$. The hard relation (Y, y) reuses exactly that structure, so a statement is "just another public key": recovering its witness means finding a short preimage of Y under A, a Module-SIS problem, while Module-LWE keeps the witness hidden [9]. *Key generation is shared* — LAS reuses the base KeyGen unchanged — so LAS is exactly the base signature plus four adaptor operations.

The base signature samples a masked commitment w, forms a challenge $c = H(pk, w, M)$, computes a response and rejects when $\|\mathbf{z}\|_\infty > \gamma - \kappa$. Relative to a statement Y, the adaptor extension [9] adds PreSign, which folds the statement into the hash, $c = H(pk, w + Y, M)$, and rejects at the *tighter* bound $\gamma - \kappa - 1$; PreVerify, which recomputes $w' = A\mathbf{z} - c\,t$ and checks $c = H(pk, w' + Y, M)$; Adapt, which adds the witness, $\mathbf{z} \leftarrow \hat{\mathbf{z}} + y$, producing a signature the base verifier accepts; and Extract, which recovers $y = \mathbf{z} - \hat{\mathbf{z}}$ (fig. 2.1).

$$\text{accept iff } \|\mathbf{z}\|_\infty \le \gamma - \kappa, \qquad \gamma = \kappa\, d\, (n + \ell). \tag{2.1}$$

The single subtle design point is the bound budget. The witness is ternary, so $\|y\|_\infty \le 1$, and because PreSign rejects at $\gamma - \kappa - 1$ the response after Adapt clears eq. (2.1); loosening it to $\gamma - \kappa$ would let adapted signatures exceed the

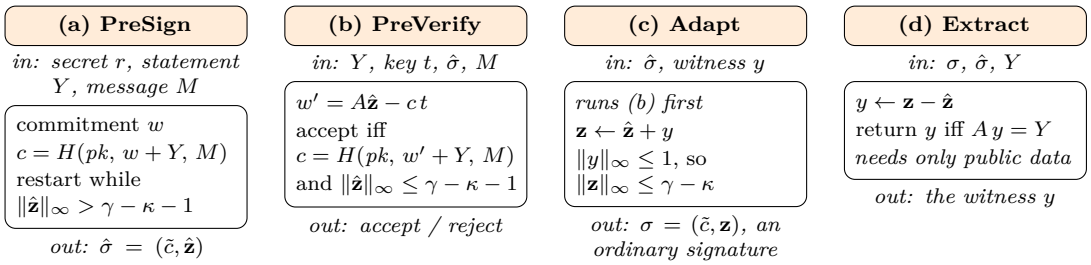

Figure 2.1: The four functions LAS adds to the base signature, one panel each: what goes in, what the function computes, and what comes out. KeyGen, Sign and Verify are reused unchanged and are not shown. Two asymmetries are the whole construction. Only (a) and (b) touch the statement Y, and only in one place — the challenge hash — which is why the adaptor layer inherits the base scheme's arithmetic rather than replacing it; only (c) and (d) touch the witness y, and they are the two functions a basic signature has no counterpart for. The bound in (a) is the one quantity that must not be relaxed: pre-signing rejects one unit tighter than eq. (2.1) precisely so that the addition in (c) cannot carry the adapted response past the bound the base verifier enforces. How the four compose, and why publishing σ is what makes a swap atomic, is fig. 2.2.

bound and be rejected. Figure 2.2 shows the data flow.

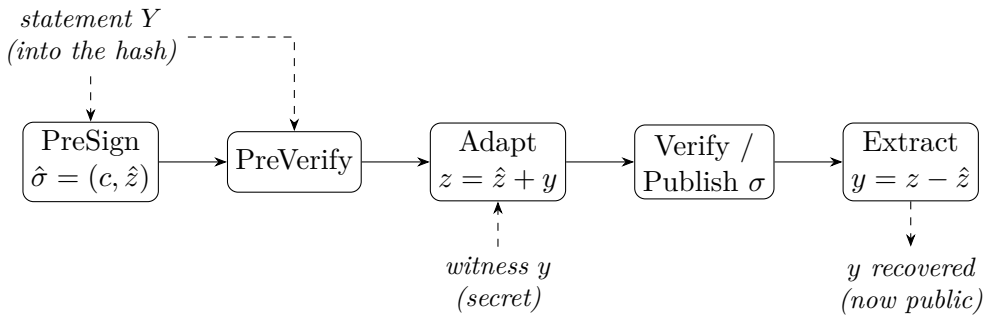

Figure 2.2: The LAS adaptor data flow. The statement Y is folded into the challenge hash during PreSign and PreVerify; the witness y is added by Adapt to turn the pre-signature into a base signature; and publishing that signature lets anyone holding the pre-signature run Extract to recover y. This last step is what makes an atomic swap atomic.

2.2 Implementation strategy: reuse, do not reinvent

The guiding principle is that an exotic scheme equals a basic scheme plus extra functions, so the lattice arithmetic should be *reused*, not rewritten. The implementation is built additively on the upstream CRYSTALS-Dilithium reference [8, 6] at a pinned commit, and the same strategy is applied again in Rust (section 2.5). Neither is itself "the reference": both are *modified, simplified* ML-DSA-style bases carrying the LAS layer of [9].

Classifying every source file by how the reference is used (table A.7) yields a clean split: the entire lattice core is reused unchanged, the only modified file is the `Makefile`, and the scheme, serialization, application and evaluation are new. No upstream function is modified, so the security-critical arithmetic is exactly the published reference code and every adaptor-specific line is isolated. The vendored `secp256k1-zkp` [4] and LaZer [14] are unmodified too, the latter behind a single-file bridge.

Data types and the matrix product. LAS is a self-contained module over the reference's degree-d polynomial type. Because $A = [\, I_n \mid A' \,]$, the product $A\,v = v_{\text{top}} + A' v_{\text{bot}}$ multiplies only A'.

Hash, challenge, and samplers. The random oracle H is built from SHAKE256, absorbing a canonical encoding of the public key, then the commitment (w for Sign, $w + Y$ for PreSign), then the message. The challenge sampler is the reference's `SampleInBall` at weight κ, giving $\|c\|_1 = \kappa$ and $\|c\|_\infty = 1$. Two samplers are added: a ternary one for secrets and witnesses, and a rejection sampler uniform on $[-\gamma, \gamma]$ for the mask (appendix A.3).

Simplifications, and the modulus. Following [9], the base signature omits Dilithium's hint vector, Power2Round and high/low-bit decomposition, hashing the full commitment w rather than its high bits. This keeps the exact identity $A\mathbf{z} - c\,t = w$ available, at the cost of larger keys and signatures. Whether those optimisations can coexist with an adaptor layer was not assumed but *tested* (section 3.5). The modulus is reused too: the reference NTT fixes $q = 8\,380\,417 \approx 2^{23}$, FIPS 204's own modulus [18], rather than the $q \approx 2^{24}$ of [9] — correctness is

unaffected since $q > 2\gamma$, and only the Module-SIS/LWE margin differs. No module depends on a mode-specific constant, so identical code runs at every parameter set.

2.3 Serialization and the on-chain interface

A deployable scheme exchanges *bytes*, not in-memory structs, so a bit-packed wire encoding, a *validating* decoder and a verify-from-bytes entry point were implemented. Two decisions matter. The decoder is *defensive*: it is the interface an on-chain verifier would expose, and the object the tamper test attacks. And, following FIPS 204 [18], a signature carries not the challenge polynomial c but the digest $\tilde{c}$ from which every consumer re-derives it, under the standard's $\lambda/4$ rule (appendix A.5).

2.4 Testing methodology

Four test types cover progressively weaker assumptions about the environment, stated in full in table A.2. Functional tests assume an honest caller and assert the whole adaptor cycle, including the *tripwire* clause — a pre-signature must *fail* basic verification — which is what distinguishes an adaptor signature from a signature. Serialization and tamper tests assume a hostile caller; the known-answer test assumes a different machine, and its digest is what the second implementation is checked against (section 2.5).

2.5 An independent second implementation: the Rust port

The complete scheme was implemented a second time in Rust on the same reuse principle, layered on a fixed version of the `fips204` crate [21], with the LAS layer added as new modules. The wire format is byte-identical to the C encoder's by construction (fig. 2.3).

The port serves three purposes. *Cross-validation*: short of a formal proof, the strongest practical argument that a cryptographic implementation is correct is a second one written against the specification and agreeing on pinned vectors. *An*

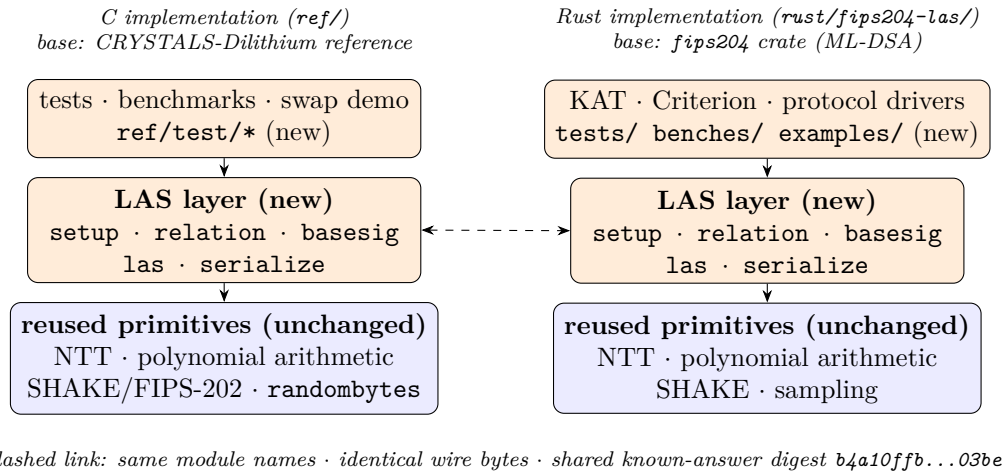

Figure 2.3: Repository structure. Both implementations have the same shape: a new, self-contained LAS layer (orange) — the same five modules under the same names — built on an *unmodified* base of reused lattice primitives (blue), with the tests and benchmarks on top. The two columns are tied together by construction: identical wire bytes and the shared pinned known-answer digest (`b4a10ffb...03be`). This is the high-level map for reading the code; the per-file classification is table A.7.

independent benchmark methodology: Criterion.rs [11] shares no timing code with the C harness, so agreement defends the C numbers against harness artefacts. *Deployment realism*: a memory-safe language is the plausible production vehicle.

To make the timings comparable the two protocol drivers mirror each other exactly — same repetition scheme, fixed seed, fixed message, success-path gating and rejection statistics. One caveat qualifies the port's independence: where [9] leaves an engineering choice open, Rust adopts the C implementation's, concretely an early-abort rejection check. Sharing that is what makes the two languages time the same algorithm, but it is established by inspecting the two norm checks rather than by the automated evidence, which constrains outputs and rejection rates, not the work profile of a rejected attempt (appendix A.3).

2.6 Benchmarking methodology and fair comparison

The evaluation separates *computation* cost (wall-clock time per operation) from *communication* cost (bytes transmitted or stored).

Measurement boundaries. Undocumented boundaries are a recognised benchmarking pitfall for lattice signatures [20], so every timing belongs to one of the three declared boundaries of table A.4: a *core tier* isolating the adaptor algebra from encoding, a *packed tier* giving what an on-chain consumer pays, and, for the classical baseline alone, the *hybrid* boundary its library exposes. Comparisons are made only within one boundary, and every base-versus-LAS comparison shows *both*.

Implementation of record, repeatability, and machine. Unless a caption says otherwise, timings are the C implementation's, with the Rust results reported separately and never averaged with them (table 3.4). Every timing is a mean $\pm$ sample standard deviation over the repetition scheme its caption states. All non-random inputs are fixed, so variability comes only from rejection sampling and machine noise, never from input selection [20]. Toolchain, hardware and per-table commands: appendix A.1.

Per-operation reporting, not cumulative. Every operation is timed independently: they are split across parties, so a combined figure would average over costs never paid together and hide the per-operation overhead.

The primary comparison: LAS against its own simplified base. LAS is exactly its base signature plus four operations on the same code, parameters and primitives, so every comparison holds all three fixed and varies only the adaptor layer (justified in section 4.1). Each adaptor operation is paired with the base operation it mirrors — Adapt against Verify, since it runs a PreVerify before adding the witness — and Extract, having no base analogue, is reported separately.

Rejection sampling: the expected attempt count. Sign-class operations restart until the response passes its norm bound, so no timing claim is meaningful without the attempt statistics. Each coefficient is accepted with probability $\big(2(\gamma - \kappa) + 1\big)/(2\gamma + 1)$ *independently of the secret* — why the response is

simulatable (appendix A.3) — and an attempt succeeds only if all $(n + \ell)\, d$ pass:

$$p_{\text{acc}} = \left(\frac{2(\gamma - \kappa) + 1}{2\gamma + 1}\right)^{(n+\ell)d} \approx \left(1 - \frac{\kappa}{\gamma}\right)^{(n+\ell)d} \approx e^{-\kappa(n+\ell)d/\gamma} = e^{-1} \approx 36.8\%, \tag{2.2}$$

the last step using the design choice $\gamma = \kappa\, d\, (n + \ell)$ from [9], made exactly so the expected number of restarts is "about $e < 3$". Expected attempts are $1/p_{\text{acc}}$, and PreSign's bound being tighter by one, it is *expected* to restart slightly more often before any adaptor arithmetic is counted.

Rejection sampling: the run-validity gate. Because the attempt count is geometric, per-call timings are heavy-tailed and averages over too few calls can drift several percent on rejection luck alone [20]. The drivers therefore treat the attempt statistics as a *validity gate*: measured mean attempts must match the closed-form expectation within five standard errors, and a failing run aborts instead of producing evidence. This is a consistency check on the rejection rate, not a proof of sampler correctness (appendices A.2 and A.3). A second gate precedes every timing: the correctness contract is asserted on the objects about to be timed, so no failure path is ever measured.

Parameter sensitivity and component sizes. To test whether the adaptor overhead scales with the lattice dimensions, the same comparison is repeated at the four parameter sets of table 2.1 (symbols in table A.3). Each key and signature is decomposed into components, so the *cause* of a size difference can be identified, not merely its total stated.

The classical baseline: functionality-matched, not security-matched. The second baseline is the `secp256k1-zkp` library's `ecdsa_adaptor` module [4], implementing Fournier's construction [10], unmodified at a pinned commit. It is a functionality-matched "price of post-quantum" baseline, not a same-security one; plain ECDSA is excluded, the fair counterpart of an adaptor signature being another adaptor signature.

Table 2.1: Parameter sets used in the evaluation. The *LAS-2020/845 reference* set is the parameter set proposed in [9], a historical paper-parameter reference point corresponding to no ML-DSA set; the *Simplified Dilithium-II/III/V* sets reuse the dimensions of Dilithium modes 2/3/5, aligning the reusable ML-DSA primitives and the challenge-digest strength $|\tilde{c}|$ with ML-DSA-44/65/87 while retaining LAS's own ternary secret distribution, rejection bounds, exact hint-free relation and unoptimised serialization. They are engineering benchmark settings, *not* formal NIST/ML-DSA security-equivalence claims: a matched dimension is not a claim that LAS *is* the corresponding ML-DSA parameter set or inherits its security category. All lattice sets share the ring degree $d = 256$ and modulus $q = 8\,380\,417 \approx 2^{23}$ inherited from the reused Dilithium NTT; *Simplified Dilithium-III* is the project's target set. The classical secp256k1 ECDSA adaptor is a functionality-matched baseline at ≈128-bit classical security. Table A.3 defines each symbol.

Setting	n	ℓ	M	κ	γ	d	q
LAS-2020/845 reference	4	4	8	60	122 880	256	8 380 417
Simplified Dilithium-II	4	4	8	39	79 872	256	8 380 417
Simplified Dilithium-III (target)	6	5	11	49	137 984	256	8 380 417
Simplified Dilithium-V	8	7	15	60	230 400	256	8 380 417
Classical (secp256k1 ECDSA adaptor)	≈128-bit classical (see table 3.6)						

2.7 Application methodology: the UTXO swap study

The narrated demonstration of section 3.6 shows that the two-party atomic-swap protocol of Esgin, Ersoy and Erkin [9] — Figure 1 of that paper, set out in appendix A.6 and called *the swap protocol* here — *runs*. A second study measures what it *costs*, and its conclusions depend on what is held fixed.

The venue, and why it is modelled. The construction [9] states its applications over "a UTXO-based blockchain like Bitcoin *where the signature algorithm is replaced with a lattice-based signature scheme*". That sentence is implemented literally: a UTXO ledger was built in which *the signature algorithm is a parameter*, so one ledger serves every configuration and any measured difference is the cryptography, not the ledger. It is a model, not a node; the boundary is set out in appendix A.4.

What a settled transaction contains, and what the adaptor layer changes. Because the venue is a transaction rather than a contract, which *components* the construction touches has a precise answer, and fig. 2.4 gives it field by field. Of the objects the swap protocol defines, exactly one reaches a ledger: the adapted signature σ, in the slot an ordinary signature already occupies — since SegWit [15], a segregated *witness* field. Everything else is an off-chain message, and the witness y is never transmitted at all, u_2 deriving it from the published σ. The adaptor layer thus adds *no on-chain field* — which is what *scriptless* means — so every on-chain byte by which the post-quantum swap exceeds the classical one is attributable to the signature scheme alone (section 3.8).

One notational point matters once the venue is a real ledger. The swap protocol writes the signed object as tx_i, but a transaction is not a message: a signature commits to its *signature hash*. Three objects are therefore kept distinct — the unsigned template exchanged off-chain, its signature hash (the message passed to PreSign), and the settled transaction — since conflating them would misreport communication cost.

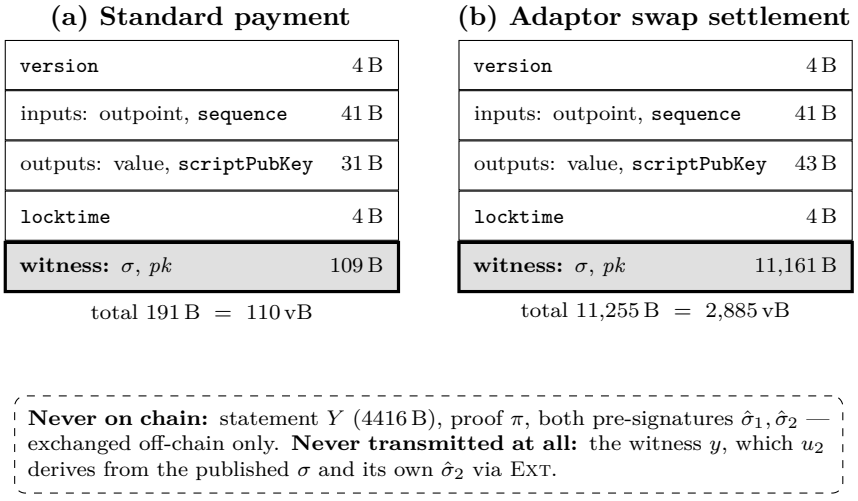

Figure 2.4: The transaction before and after the construction is applied. **No field is added, removed or reordered**: the adaptor layer is invisible on chain, which is what *scriptless* means. Only the two shaded quantities differ, and they differ because the signature scheme changed — the witness grows from 109 B to 11,161 B, and the output script from 31 B to 43 B because it commits to a 32-byte hash of a lattice key rather than a 20-byte hash of an elliptic-curve one. Sizes are the measured objects laid out in the witness serialisation of [13], with the output forms of [15, 23]; table 3.10 breaks them down field by field.

Three configurations, and what is controlled. The three configurations of table 2.2 each run the swap protocol end-to-end across two ledgers. The *role-A* proof is the proof of knowledge π the protocol places on the wire immediately after Y, claiming a witness to Y exists and is short. Only the $2 \to 3$ step is controlled: both LAS configurations share one expansion of the public matrix, derive all key material from one pinned seed, and prove the *identical* relation $\exists\, r' : Ar' = t' \wedge \|r'\|_\infty \leq 1$ — the hard relation of the statement/witness pair, *not* the signer's key pair, and including the norm bound, since proving only $Ar' = t'$ would fail to close the knowledge gap. This is verified rather than assumed: a configuration whose two halves disagree on a *relation binding* each computes independently refuses to run (appendix A.3). The $1 \to 2$ step is not controlled — it changes the signature scheme, the relation and whether a role-A proof exists — so it is a whole-stack comparison, the signature-only question being answered by section 3.4.1.

Table 2.2: The three configurations. Only the $2 \to 3$ step is controlled: those two share one public matrix, prove the identical relation, and receive byte-identical inputs from one pinned seed, so their difference is the proof system alone. Configuration 1 carries *no* role-A proof, and this is a finding rather than an omission: LAS needs π because of its knowledge gap — extraction returns a witness of the extended relation, which need not be ternary, so without π a malicious u_1 could claim and leave u_2 holding an unadaptable pre-signature — whereas an ECDSA adaptor has no such gap, since extraction recovers the discrete logarithm exactly. Deployed classical swaps accordingly carry no π, and adding one would have benchmarked a construction nobody runs. The DLEQ proof carried *inside* a `secp256k1-zkp` pre-signature is a different object and is not counted as π: its author is the pre-signer, and its claim is pre-signature well-formedness, not knowledge of the role-A statement witness r'. Its cost is reported in the bytes by which a pre-signature exceeds a signature.

#	Signature	Role-A proof π	Setup	Fully PQ
1	ECDSA adaptor signature (libsecp256k1-zkp)	none required	—	no
2	LAS (simplified Dilithium-III)	Groth16 over BN254 (pairing-based zk-SNARK)	trusted	no
3	LAS (simplified Dilithium-III)	LaZer (LNP22 linear relation with norms)	transparent	yes

The Groth16 circuit. No pairing-based proof of this lattice statement was available to reuse, so the circuit is new, and cheaper than the statement suggests: $r' \mapsto Ar'$ is *linear with public coefficients*, needing no witness–witness multiplication, the expensive ingredient in a rank-one constraint system (appendix A.3).

Measurement. Because gas does not exist on this venue, the axes are execution time and communication cost, the latter counting *off-chain* protocol messages, not only what reaches a ledger. Sizes are observed from the transmitted buffers rather than computed, and these timings sit at the **packed tier**, so they are not comparable with core-tier figures elsewhere. One-time costs are excluded and reported separately (appendix A.3).

Chapter 3

Results and discussion

This chapter reports what was measured and why the results should be believed: correctness, computation cost, communication cost, and the application results.

3.1 Correctness and robustness

No performance claim means anything until the implementation is shown *correct*, and an adaptor signature is not correct merely because it signs and verifies — it must satisfy a specific contract. Table 3.1 lists every clause; all pass, with zero compiler warnings. Clauses 1–4 are the adaptor contract itself: a pre-signature is verifiable yet unspendable (the tripwire), becomes an ordinary signature once the witness is added, and completing it reveals that witness — the mechanism that makes an atomic swap atomic. Clauses 5–6 are robustness, clause 7 reproducibility.

3.2 Computation cost

The primary computation result is the cost of the *adaptor layer*. The two paths share algorithm, parameters and primitives, so any difference is purely the price of adaptor functionality (section 2.6). Table 3.2 reports it at *both* boundaries and fig. 3.1 visualises the core tier: every adaptor operation stays within single digits of its counterpart, Extract is the cheapest of all, and every operation completes in well under two milliseconds, signing and pre-signing dominating because of rejection sampling.

At the core boundary the adaptor layer is almost free, and the reasons are structural. PreSign (+6.6% over Sign) runs the same reject loop, adding only Y

Table 3.1: The adaptor-signature correctness contract, each clause checked and passed by `test_contract` (corroborated at scale by the 1000-iteration functional suite on modes 2/3/5, the 256-instance serialization suite, and the pinned known-answer test).

#	Property	Result
1	PreSign produces a pre-signature that PreVerify accepts	pass
2	the pre-signature *fails* basic Verify (statement-binding tripwire)	pass
3	Adapt yields a signature that basic Verify accepts	pass
4	Extract recovers the witness *exactly* ($y' = y$)	pass
5a	a tampered message fails Verify	pass
5b	every one of the 6736 single-byte flips of the packed signature is rejected	pass
6	malformed packed bytes are rejected ($pk \geq Q$, ternary code-3, z out of band)	pass
7	the deterministic API is byte-identical on re-run	pass

Table 3.2: The primary computation result: per-operation adaptor overhead at the target setting *Simplified Dilithium-III* (n=6, ℓ=5, κ=49, γ=137 984, d=256), at *both* measurement boundaries of section 2.6. Each LAS adaptor operation is paired with the basic operation it mirrors; "Overhead" is the extra cost as a percentage of that basic operation, within the same tier. All timings are measured on the C implementation, the implementation of record (section 2.6); the Rust port's independent measurements are in table 3.4. Timings are μs per operation (mean $\pm$ sample standard deviation over 5 repetitions $\times$ 500/1000 iterations; machine of record in section 2.6). KeyGen, Sign, and Verify are reused unchanged by LAS; Extract has no basic analogue. The packed-tier overheads are larger because each adaptor operation additionally decodes the public-key-sized statement Y (4416 B) — serialization, not adaptor arithmetic. Absolute core-tier timings at every parameter set are in table A.5 (appendix).

Operation (basic / adaptor)	Basic (μs)	LAS adaptor (μs)	Overhead
Core tier (structures in/out — isolates the adaptor arithmetic)			
KeyGen	50.9 ± 0.9	50.9 ± 0.9 (shared)	—
Sign / PreSign	489 ± 17	522 ± 32	+6.6%
Verify / PreVerify	111 ± 3	115 ± 2	+4.0%
Verify / Adapt	111 ± 3	118 ± 2	+6.9%
Extract	—	43.7 ± 0.7	(LAS only)
Packed tier (wire bytes in/out — incl. validating decode + encode)			
KeyGen	167 ± 0.25	167 ± 0.25 (shared)	—
Sign / PreSign	732 ± 25	883 ± 19	+20.7%
Verify / PreVerify	362 ± 6	500 ± 3	+38.3%
Verify / Adapt	362 ± 6	687 ± 18	+90.0%
Extract	—	506 ± 1	(LAS only)

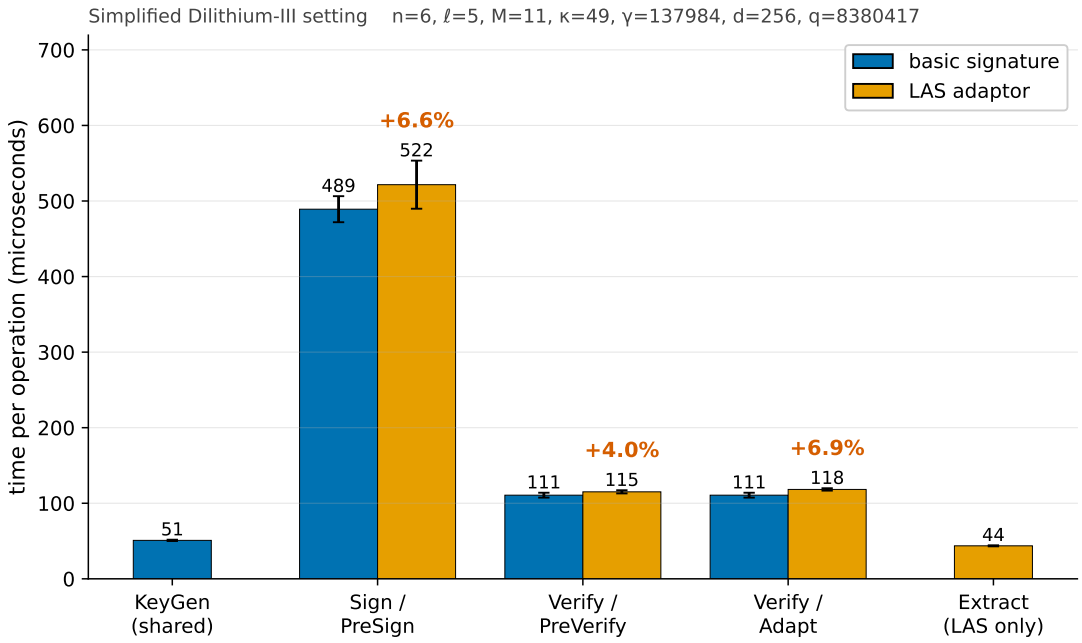

Figure 3.1: The primary computation result, shown visually: each LAS adaptor operation (orange) paired beside the basic operation it mirrors (blue), at the target setting *Simplified Dilithium-III* (its parameters are annotated above the plot), measured on the C implementation (the implementation of record, section 2.6). The percentage above each orange bar is the adaptor overhead over its blue partner; exact means and standard deviations, and the packed tier, are in table 3.2. KeyGen, Sign, and Verify are reused unchanged by LAS, so KeyGen is shown once (shared) and Sign/Verify are the blue partners of PreSign/PreVerify/Adapt; Extract has no basic analogue and is shown LAS-only. The absolute μs scale also shows the shape: every operation is sub-two-milliseconds and the rejection-sampling operations (Sign, PreSign) dominate, while the verify-class operations and Extract are far cheaper. Error bars are the sample standard deviation over 5 repetitions of 500 (sign-class) / 1000 (verify-class) iterations on the machine of record (section 2.6); the same comparison across all four parameter sets is the scaling check in fig. 3.2.

to the commitment and a tighter bound; PreVerify (+4.0%) recomputes the same commitment shape with the extra $+Y$; Adapt (+6.9%) runs a PreVerify then adds the witness vector; Extract merely subtracts $z - \hat{z}$ and checks $A\,y = Y$. None changes the asymptotic work. This is the headline of the standalone-signature evaluation: *at the core tier* adaptor functionality costs single-digit percentages at every parameter set, and the two operations unique to an adaptor signature cost no more than a verification each (fig. 3.2).

At the packed boundary the premium grows to +20.7%, +38.3% and +90.0%

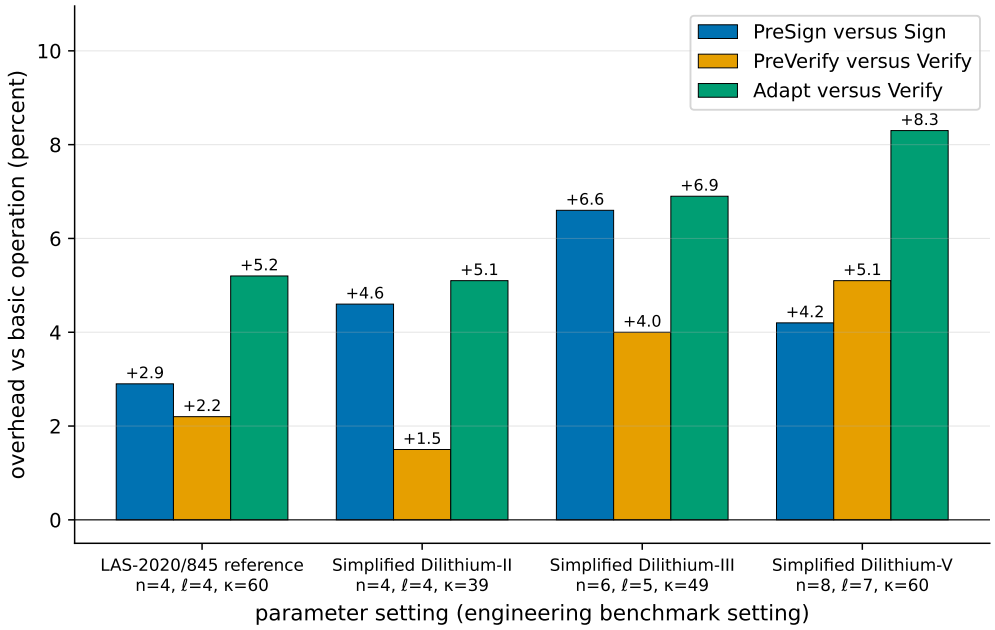

Figure 3.2: Parameter sensitivity of the adaptor overhead: each LAS adaptor operation as a percentage over the basic operation it mirrors, at the four parameter sets of table 2.1. PreSign is paired with Sign; PreVerify and Adapt with Verify; Extract has no basic analogue and is omitted. The core-tier premium stays in the single digits everywhere, so adding adaptor functionality does not become more expensive as the scheme is scaled up. This is a secondary robustness check on the headline of fig. 3.1, not a result about scaling: the primary comparison is the per-operation overhead at the target setting, table 3.2. The absolute timings behind every bar are table A.5 (appendix).

— an order of magnitude higher, and the gap is the statement's decode rather than adaptor arithmetic. The object dominating *communication* cost thus reappears as the dominant *computation* cost once bytes are included, so a deployment verifying from bytes should budget for the packed row — and reported lattice timings are notoriously sensitive to this boundary [20].

Signing and pre-signing are dearer than verification because of rejection sampling, intrinsic to the Fiat–Shamir-with-aborts family rather than a defect here. Equation (2.2) predicts 2.719 attempts per Sign and 2.775 per PreSign, the latter restarting about 2% more often because its bound is tighter by one; counted directly rather than estimated from a timing ratio, the run of record observed 2.615 and 2.724 (table 3.3). At 2500 timed calls the gate band ($\pm\sim 8\%$) is wider than that 2% separation, so the core-tier PreSign overhead (+6.6%) should be read as of the same order as the prediction rather than as a precise constant; only

Table 3.3: Rejection-sampling statistics at the target setting *Simplified Dilithium-III*: the closed-form geometric model of eq. (2.2) against every measured sample. "Mean" is attempts per call and "Accept." is the per-attempt acceptance rate — the two quantities every source records, and the ones the model predicts. The per-call *dispersion* (the geometric standard deviation $\sigma = E\sqrt{1 - 1/E}$, the medians and 95th percentiles, and the sampled maxima) is annotated on the companion acceptance curve of fig. 3.3 rather than repeated here. The C driver is the implementation of record; the Rust protocol driver and the Criterion harness are the independent samples of section 2.5. Every measured row passes the five-standard-error validity gate of section 2.6.

Operation	Source (calls)	Mean	Accept.
Sign	geometric model, eq. (2.2)	2.719	36.8%
	C driver, distribution sample (2000)	2.615	38.2%
	C driver, timed-run gate (2500)	2.756	36.3%
	Rust driver, timed-run gate (2500)	2.704	37.0%
	Rust Criterion, gate (188791)	2.735	36.6%
PreSign	geometric model, eq. (2.2)	2.775	36.0%
	C driver, distribution sample (2000)	2.724	36.7%
	C driver, timed-run gate (2500)	2.852	35.1%
	Rust driver, timed-run gate (2500)	2.766	36.1%
	Rust Criterion, gate (188791)	2.784	35.9%

the $\approx 10^5$-call Criterion harness resolves it.

3.3 Cross-implementation check: C implementation versus Rust port

The scheme exists twice (section 2.5), and the two implementations agree at every level at which they can be compared. At the byte level agreement is exact: the same packed sizes (public key 4416 B, signature 6736 B, checked programmatically on every Rust run) and the same pinned known-answer digest (`b4a10ffb...03be`). Since that digest covers key generation, signing, the four adaptor operations and the serialization, a divergence anywhere in either implementation's sampling, algebra or encoding would flip it.

Timing agreement is looser, the builds going through different compilers, but close: every operation agrees within about 18% (largest gap Adapt). More importantly the central conclusion reproduces in both languages. At the core tier the Rust port measures $+2.9\%$, $+-0.7\%$ and 1.8% — all single-digit, like the C

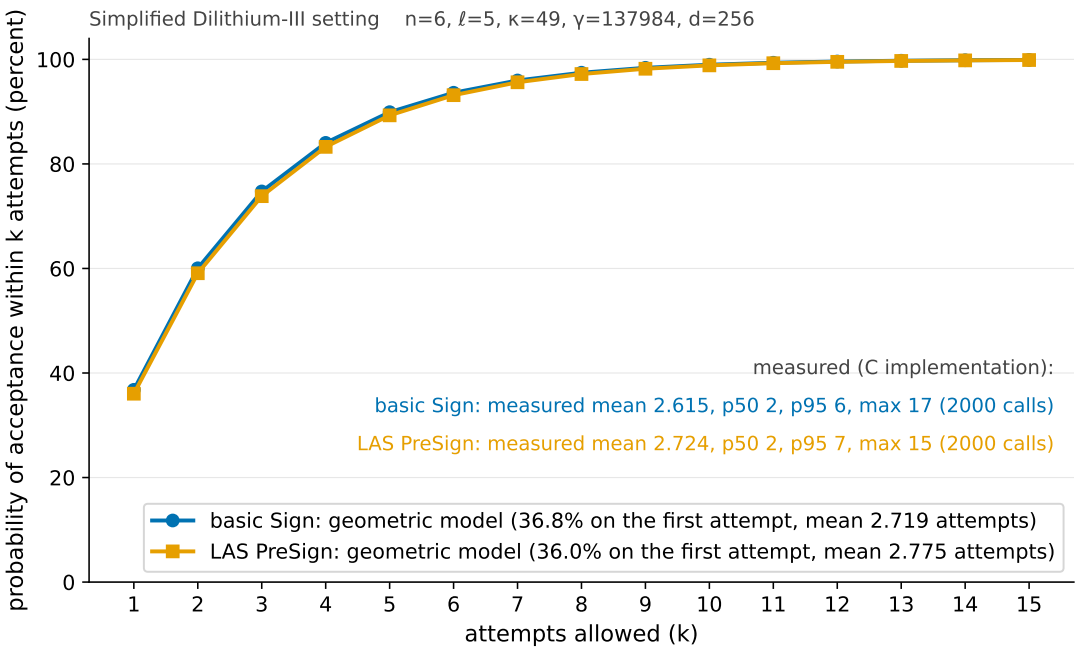

Figure 3.3: The probability that a sign-class call has been accepted *within k* rejection-sampling attempts, at the target setting *Simplified Dilithium-III*. The curves are the closed-form geometric model of eq. (2.2) for the basic Sign bound $\gamma - \kappa$ (blue) and the tighter LAS PreSign bound $\gamma - \kappa - 1$ (orange): acceptance is 36.8% and 36.0% respectively on the first attempt, passes 90% by five attempts, and then flattens — the two curves are nearly coincident, which is exactly why the PreSign-versus-Sign timing gap is so small. The annotated statistics are *measured* on the C implementation (the 2000-call distribution sample tabulated exactly in table 3.3). The flattening is the practical point: allowing many more attempts buys almost nothing, because the overwhelming majority of calls are accepted within the first few.

column beside them; the slightly *negative* Adapt figure is the measurement floor rather than a real speed-up, the genuine overhead at this scale being smaller than the gap between the two verify paths. The packed tier reproduces too (+5.3%, +16.8%, +32.1%), all positive and in the C figures' order. Both ports pass the same rejection gate (fig. 3.4). The headline results are therefore properties of the algorithm, not of one codebase, compiler or harness.

Table 3.4: Per-operation timing of the two implementations at the target setting, *Simplified Dilithium-III* (μs per operation). The C and Rust columns come from the two protocol drivers, which deliberately mirror each other (same repetition scheme of 5 repetitions $\times$ 500/1000 iterations, fixed public-parameter seed, fixed message, same success-path gating), so they are directly comparable. The final column is the independent Criterion.rs statistical harness (300 samples per operation, bootstrap 95% confidence interval of the mean); its hot-loop protocol yields lower absolute times, and it is included to show that the *relative* conclusions do not depend on the hand-rolled driver.

Operation	C implementation (μs)	Rust port (μs)	Rust / C	Rust, Criterion (μs, 95% CI)
KeyGen	50.9 ± 0.9	51.5 ± 0.5	1.01	41.3 [41.2, 41.4]
Sign	489 ± 17	438 ± 14	0.90	377 [375, 378]
Verify	111 ± 3	95.4 ± 0.5	0.86	75.6 [75.3, 75.9]
PreSign	522 ± 32	451 ± 7	0.86	386 [384, 388]
PreVerify	115 ± 2	94.8 ± 0.9	0.82	76.8 [76.4, 77.1]
Adapt	118 ± 2	97.1 ± 1.4	0.82	77.9 [77.5, 78.3]
Extract	43.7 ± 0.7	37.8 ± 0.2	0.87	28.5 [28.4, 28.6]

3.4 Communication cost and component breakdown

The computation results show the adaptor layer is cheap at the core boundary; the size results show where the real cost lies. Table 3.5 decomposes every LAS object into bytes at the target setting, explaining *which* component drives the size. Sizes are fixed by the encoding, so they are exact and carry no dispersion.

Two points follow. First, **the response z is essentially the entire signature** — 99.3% of it at every parameter set — since the challenge travels only as its short digest, so any optimisation of LAS signatures is an optimisation of z. Second, **an adaptor swap additionally transmits a public-key-sized statement Y and a signature-sized pre-signature**: the only objects an adaptor scheme sends that a basic signature does not.

The sharp conclusion: the cost of LAS is not adaptor computation but communication — specifically z and Y. The adaptor operations add at most +8.3% of compute, while the objects on the wire are kilobytes. That matters more in a blockchain setting than "the signature is 6736 bytes": bytes reaching a ledger are stored and replicated by every node, while the statement and pre-signature, though off-chain, still cost both parties bandwidth. The response is stored at full

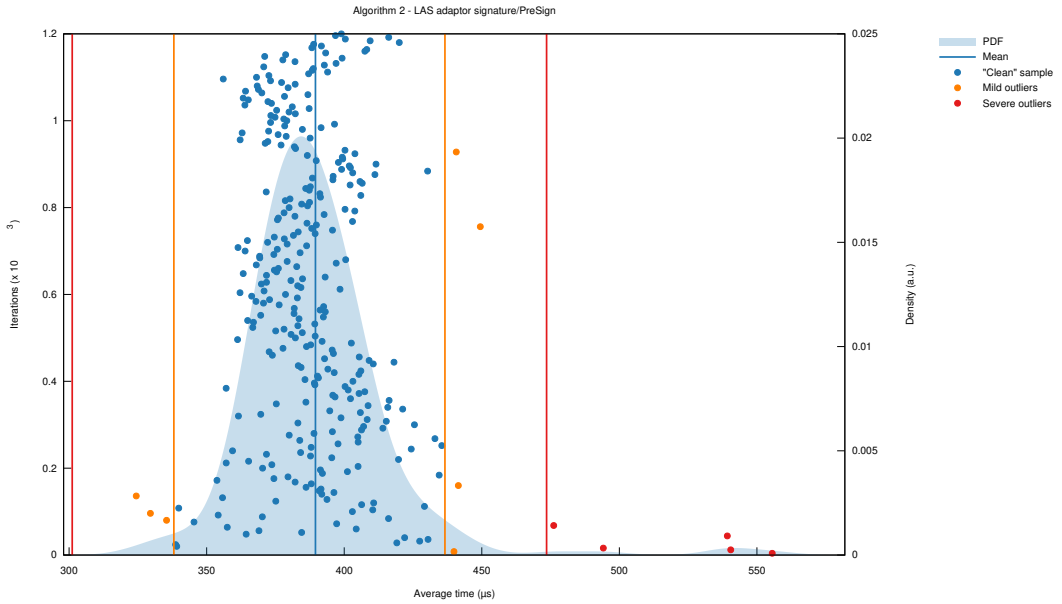

Figure 3.4: The statistical harness's own report for PreSign at the target setting, reproduced unmodified from Criterion.rs (its SVG output converted to PDF): 300 timed samples (dots), the kernel-density estimate of the sampling distribution (shaded), the mean (vertical blue line), and Criterion's automatic outlier classification (mild / severe, orange / red). This is the evidence behind the Criterion column of table 3.4: a well-formed, single-mode distribution whose spread comes from rejection sampling and scheduling noise, with only a handful of classified outliers.

width, neither range-reduced nor entropy-coded — the clearest opportunity for reducing that footprint (section 5.3).

3.4.1 The price of post-quantum: classical adaptor baseline

The second baseline is a classical secp256k1 ECDSA adaptor signature [4]: the natural functional counterpart of LAS, driving the same scriptless swaps but resting on discrete-log rather than lattice hardness. It measures the practical cost of moving from a classical adaptor signature to a post-quantum one. LAS is instantiated at *Simplified Dilithium-II* and only there: secp256k1 offers roughly 128-bit classical security, so those dimensions are the most like-for-like engineering match, while -III or -V would pit the classical scheme against a deliberately stronger configuration. The boundaries differ between the two libraries, so table 3.6 matches them per operation and the comparison is read conservatively — the conclusions rest on order-of-magnitude gaps, not percent-level differences.

Table 3.5: Serialized size of every LAS object at the *Simplified Dilithium-III* (target) setting, in packed wire bytes (not on-chain gas), with its paper notation. "In the basic signature?" marks which objects LAS shares with the base scheme and which it adds. Three facts read off directly: the signature, pre-signature, and adapted signature are byte-identical (adaptation changes the *value* of the response, not its size); the response z dominates every signature; and the only extra *public* object LAS adds over a basic signature is the statement Y, the size of a public key. The sizes hold for the C and the Rust implementation alike: the wire encoding is byte-identical in both by construction, asserted programmatically on every Rust run (section 3.3). Sizes at every parameter set: table A.6 (appendix).

Object	Notation	Bytes	In the basic signature?
Public key	$pk = t$	4416	yes (shared)
Secret key	$sk = r$	704	yes (shared)
Challenge	c	48	yes
Response	z / $\hat{z}$	6688	yes
Signature	(c, z)	6736	yes (basic signature)
Statement	$Y = t'$	4416	LAS only (public)
Witness	$y = r'$	704	LAS only (private)
Pre-signature	$(c, \hat{z})$	6736	LAS only
Adapted signature	(c, z)	6736	LAS (verifies as basic sig)

Two findings stand out. First, **the post-quantum price is paid almost entirely in size**: LAS's signature and public key are roughly 72× and 89× larger than the classical adaptor's, while every LAS operation stays well under a millisecond. Even the largest per-operation factor (Adapt, 286×) repays decomposition, because it compares two different *quantities of work* rather than two speeds for the same work: Algorithm 2 of [9] obliges Adapt to run PreVerify before returning, so 69% of LAS's 443 μs is a full lattice verification and the rest is largely codec, whereas `secp256k1_ecdsa_adaptor_decrypt` merely inverts one scalar and multiplies, leaving verification to a separate exported call [4]. Charging the classical side the verification LAS cannot skip puts the two at 119 μs against 443 μs — 4×, not 286×: a property of the two *interfaces*, not of lattice arithmetic. Second, **the two schemes pay their adaptor overhead in opposite ways**. The classical adaptor must attach a discrete-logarithm-equality proof [10], making its PreSign and PreVerify several times its own base operations, whereas LAS folds the statement into a hash it already computes. The lattice adaptor layer is the cheaper one to add.

Table 3.6: Classical vs. post-quantum adaptor signature: same functionality, different security foundation. Classical secp256k1 (≈128-bit *classical* security) against LAS at the *Simplified Dilithium-II* parameters of table 2.1. Those dimensions are an engineering match to the ≈128-bit (NIST level 2) target, *not* a formal security-equivalence claim; the stronger LAS settings are reserved for the parameter-sensitivity study (section 3.2) and would overstate the post-quantum cost here. Timings μs/op (mean ± SD, machine of record); sizes in bytes. The two libraries draw their measurement boundaries differently, so the final column is *tier-matched* — each LAS row taken at whichever of its two tiers corresponds to the classical library's boundary for that operation. Why that matching is necessary, the boundaries themselves, the repetition counts, and the object-level differences behind the size rows are set out in appendix A.3.

	ECDSA adaptor (classical, hybrid native API)	LAS adaptor (post-quantum, core tier)	LAS adaptor (post-quantum, packed tier)	overhead LAS ÷ classical (tier-matched)
Computation (μs/op)				
KeyGen	14.4 ± 0.03	34.1 ± 0.1	116 ± 1	2.4×
Sign	19.9 ± 0.1	314 ± 14	494 ± 4	16×
Verify	29.6 ± 0.3	74.1 ± 0.7	134 ± 18	2.5×
PreSign	91.2 ± 1.2	328 ± 19	576 ± 11	6.3×
PreVerify	117 ± 1	75.2 ± 0.6	305 ± 8	2.6×
Adapt	1.6 ± 0.02	77.8 ± 0.2	443 ± 3	286×
Extract	16.0 ± 0.1	28.4 ± 0.1	323 ± 5	20×
Communication (bytes)				
Public key / statement	33	2944		89×
Secret key / witness	32	512		16×
Signature	64	4640		72×
Pre-signature	162	4640		29×

3.5 Testing the simplification: the adaptor on unmodified ML-DSA

Section 2.2 disabled Dilithium's hint vector, Power2Round and high/low-bit decomposition, reasoning that the adaptor needs the exact identity $A\mathbf{z} - c\,t = w + Y$. That was an *assertion*, so it was tested: the four adaptor operations were built a third time on NIST ML-DSA *as FIPS 204 specifies it* [18], all three optimisations enabled. Three findings follow; the second overturns the assertion.

The naive port fails outright. Adding the statement to the challenge hash and changing nothing else — treating the hint machinery as a black box —

yields pre-signatures that do not pre-verify at all (0/200), because the transmitted hint reconstructs HighBits(w) while the challenge committed to HighBits($w + Y$). Some modification is therefore genuinely unavoidable.

But the verifier is not what must change. Once the *whole* commitment path moves onto $w + Y$ — the committed high bits, the low-bits rejection test *and* MakeHint — and PreSign tightens its bound by the witness norm, the construction satisfies the full adaptor contract (13/13 items, including four tamper rejections and byte-identical determinism), and, decisively, **the unmodified FIPS 204 verifier accepts the adapted signature** (200/200). So PreSign and PreVerify are necessarily new algorithms, while Verify is not: a post-quantum adaptor swap could settle against a standard ML-DSA verifier. A matched no-statement baseline reproduces FIPS 204's own published repetition rates, which is what makes these rows attributable to the adaptor rather than to a porting error.

The size gain is real but bounded, and it relocates the bottleneck. Measured against the scheme of record in one process, the adaptor layer stays single-digit on both constructions (+2.8% against +2.2% for PreSign per attempt), and per-signature cost is close to equal for opposite reasons: ML-DSA restarts about twice as often (5.2172 against 2.7136) but each attempt is roughly half the price. What ML-DSA buys is *bytes* — the signature falls to 3309 B from 6736 B — but the statement Y does not shrink at all (byte-identical at 4416 B), because Power2Round compresses a public key while Y enters the verification identity *before* any rounding. The swap payload therefore improves only to 0.69×, and Y becomes larger than the signature it accompanies. **Compressing the signature moves the bottleneck to the statement rather than removing it**, redirecting the size question of section 3.4 and the future work of section 5.3.

This is a functional demonstration, not a security argument: whether committing to HighBits($w + Y$) preserves ML-DSA's unforgeability is not analysed, and is exactly the kind of claim section 4.4 declares out of scope.

3.6 Application demonstration

The implementation drives an end-to-end post-quantum atomic swap following the swap protocol [9] *verbatim*, including its proof of knowledge π (fig. 3.5 and appendix A.6). Two results matter. First, the protocol runs and every step is

asserted, including the counterfactuals that a pre-adaptation signature fails basic verification and that π is rejected against any other statement. Second, π is what makes the second party's final step *guaranteed* rather than merely expected: by the Module-SIS uniqueness argument of [9] the extracted witness equals the proven ternary one, so the adapted signature is certain to clear the verification bound — without π a malicious counterparty could claim and leave an unadaptable pre-signature behind. The proof measures 32 046 B at a knowledge error below 2^{-127} and is exchanged off-chain only.

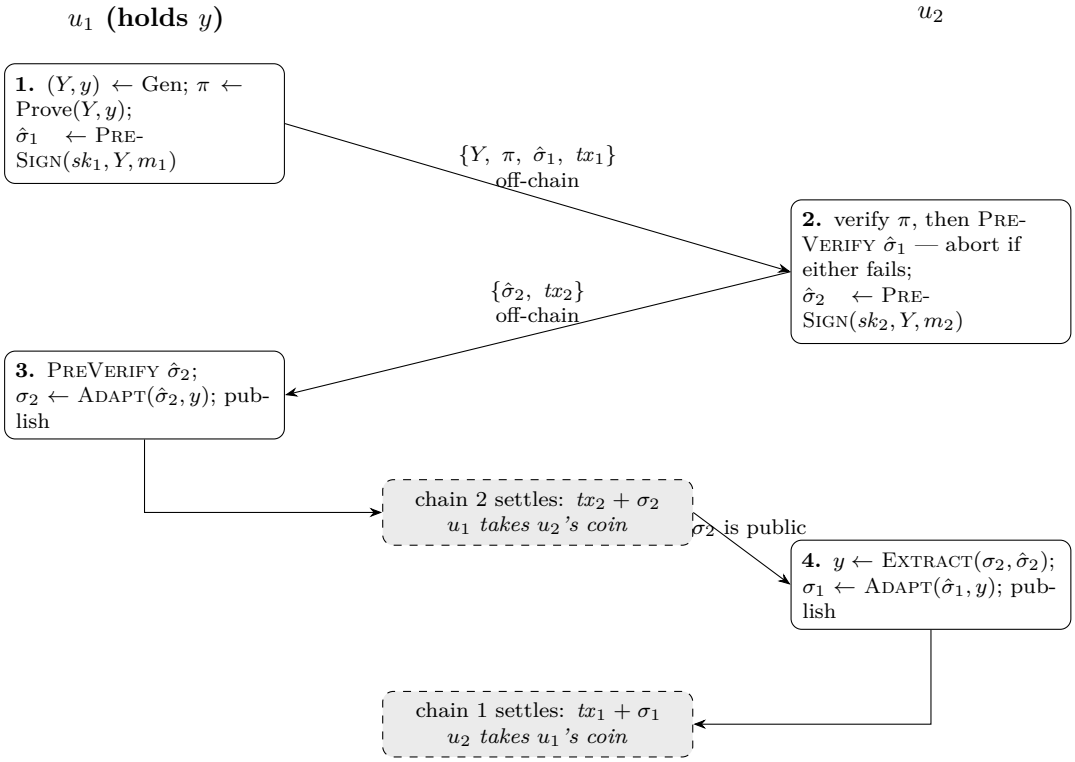

Figure 3.5: The swap protocol as implemented, with every object the two parties exchange. u_1 generates the statement/witness pair, so it moves first and claims first. The signed message m_i is the signature hash of the unsigned template tx_i, not the transaction itself (section 2.7). Step 2 is the abort gate: u_2 commits nothing until both π and u_1's pre-signature verify, and at that point neither pre-signature is spendable by anyone. Step 4 needs no message from u_1 at all — u_2 reads σ_2 off chain 2 and subtracts its own pre-signature from it, and that leak is what makes the exchange atomic rather than merely simultaneous. Only the two dashed boxes reach a ledger: the statement, the proof and both pre-signatures never do, and the witness y is transmitted by nobody. The two off-chain arrows and the two dashed boxes are exactly the message rows of table 3.9, and the operations named in the boxes are the phases timed in table 3.8.

As a supporting check on deployability, native on-chain LAS verification was *implemented* and measured on a real Solidity stack: one claim transaction running full verification and releasing one leg's escrow costs ≈56.6 million gas (table 3.7; fig. 3.6), ≈748× the classical claim and over the EIP-7825 cap. An *optimistic* (Naysayer) alternative makes the honest path cheap (≈1.2 million) but leaves a worst-case fraud proof at ≈28.2 million, also over the cap, and a fraud proof that cannot be mined is not one (appendix A.7). These measurements are why the application was taken to a UTXO chain.

That figure measures one *implementation*, not the scheme. A restructured verifier — same parameters, bounds and challenge preimage, differing in how the computation is expressed — was mined by a real client, running full LAS verification and releasing the escrow at 16 413 275 gas: 97.8% of the EIP-7825 cap, which bounds the transaction gas limit and so covers calldata as well as execution. Its equivalence is conditional on a registration invariant, and both are set out in table 3.7. On-chain verification therefore fits in one transaction at *Simplified Dilithium-III* for the 32-byte signed message measured, with a thin margin that is a message-length budget rather than slack. The change of venue stands: the UTXO constraint is a transaction *weight* limit one settlement sits well inside (section 3.8).

3.7 The swap on a UTXO chain: three configurations compared

The demonstration above establishes that the protocol runs; this section measures what it *costs*, over the UTXO ledger and under the design of section 2.7: three configurations (table 2.2), of which only the $2 \to 3$ step is controlled. Timings are the mean ± sample standard deviation over 10 complete swaps at the *packed* tier, so they are not comparable with section 3.2. Both LAS configurations passed the rejection gate (2.8510 attempts per PreSign against the closed-form 2.77483).

The proof, not the signature, is the cost of a post-quantum swap. The role-A proof consumes 99.3% of end-to-end time in configuration 2 and 98.6% in configuration 3. Strip it out and the entire post-quantum signature workload — two key generations, a statement, two pre-signatures, a pre-verification, two adaptations, an extraction and two settlements — costs 4468.6 μs, about 5.5×

Table 3.7: On-chain gas for the swap's claim step, measured on a local EVM (`forge -gas-report`); EVM gas is deterministic for fixed bytecode, revision, inputs and state. **These are *execution* figures**: they cover the work done inside the call, but not the 21,000 intrinsic charge or the per-byte calldata cost a real transaction also pays under EIP-7623 — which is why the under-the-cap claim below rests on a client receipt rather than on this table. `claimLAS` is a floor performing *no* lattice verification (calldata plus one `keccak256`), a strict lower bound. The two `claimLASVerified*` rows decide the same predicate as `base_verify` (section 2.1); the second *restructures* that computation rather than changing it, and is validated against both the first and the C reference (appendix A.3). ECDSA, by contrast, verifies in the native `ecrecover` precompile, whereas both LAS rows run in EVM bytecode. **One condition qualifies that equivalence**: the restructured verifier consumes a registered $\hat{t}$ rather than deriving it, so the two agree where $\hat{t} = \mathrm{NTT}(t)$ for the key whose bytes are hashed; the fund-time commitment binds both representations against substitution but does not *prove* their consistency, which remains the registrant's obligation. The separate client run reported in the text was made on anvil at a pinned `osaka` revision with EIP-7825 enforced, over 50 148 B of calldata. **Scope**: *Simplified Dilithium-III*, a 32-byte signed message, that revision; no other parameter set was evaluated, the verifier's dimensions being fixed at build time. The 363 941 gas of headroom is *measured*; that it corresponds to a few hundred further bytes of signed message is *derived* from it and the measured per-permutation hash cost.

Claim path	On-chain verification	Gas	× classical
Classical ECDSA adaptor (`claimClassical`)	full, `ecrecover` precompile	75 763	1×
LAS floor (`claimLAS`)	none (calldata + `keccak256`)	290 640	3.8×
LAS full verify (`claimLASVerified`)	full `base_verify`, in Solidity	56 647 414	748×
LAS full verify, restructured (`claimLASVerifiedOpt`)	full `base_verify`, in Solidity	16 438 275	217×

the classical swap's 819.7 μs. A factor of five on a millisecond-scale workload is unremarkable; a proof of hundreds of milliseconds decides whether the protocol feels instantaneous. Even at the packed tier the signature workload is not the bottleneck — the post-quantum penalty is not where intuition puts it.

The controlled comparison inverts the expected trade-off. Replacing Groth16 with LaZer — same signature, same $\mathbf{A}$, same inputs — makes the swap *faster* by 53.1% while making it *larger* by 61.9%: roughly twice as quick to produce a proof, yet emitting one about 240× bigger. Groth16 is *succinct* — three group

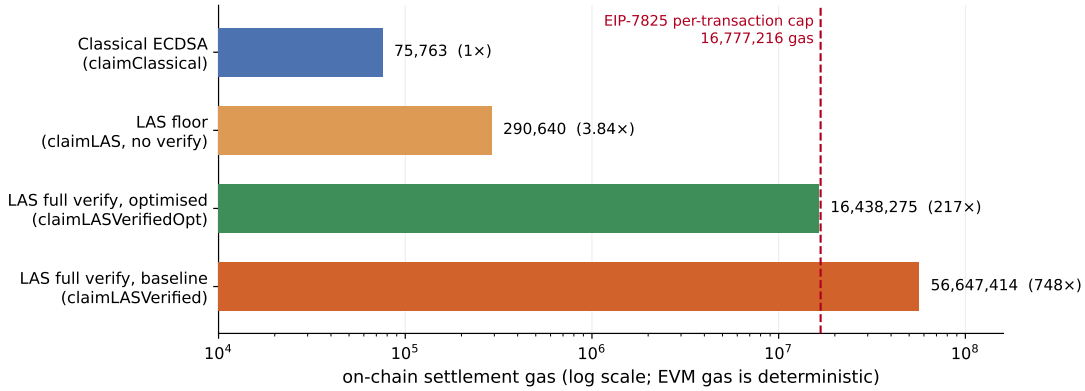

Figure 3.6: The claim paths of table 3.7 on a logarithmic axis, which is what makes the spread legible: the classical claim, the LAS floor doing *no* lattice verification, and the two full verifiers. The dashed line is the EIP-7825 cap on the transaction gas limit ($16\,777\,216 = 2^{24}$). The baseline verifier's bar crosses it; the restructured `claimLASVerifiedOpt` — same scheme, same predicate, different expression — falls below, which is what changes the conclusion from "a deployable path needs a precompile, a succinct proof or an optimistic scheme" to "at this parameter set and message length it fits, narrowly". All four bars come from one `-gas-report` table and so are mutually comparable; the under-the-cap claim itself rests on the separate real-client run described in table 3.7.

Table 3.8: Per-phase execution time of one complete swap, mean ± sample SD over 10 swaps (μs; run 20260730_162109, AMD Ryzen 7 7745HX with Radeon Graphics). Dashes mark phases a configuration does not have. End-to-end is context, not the headline.

Phase	Classical	LAS + Groth16	LAS + LaZer
KeyGen ×2 + Gen	58.4	540.7	503.2
Prove (π)	—	645621.6	216080.6
PreSign (u_1)	114.9	1065.3	1065.7
ProofVerify (π)	—	13718.1	90720.8
PreVerify (abort gate)	147.6	364.9	380.2
PreSign (u_2)	107.1	1070.1	966.7
Adapt (u_1)	138.4	392.5	380.5
Settle chain 2	51.9	253.0	239.9
Extract (u_2)	25.3	185.2	183.6
Adapt (u_2)	132.7	353.8	340.9
Settle chain 1	43.4	243.0	234.9
end-to-end	819.7	663808.3	311097.1
of which π	—	99.3%	98.6%
excluding π	819.7	4468.6	4295.7

Table 3.9: Communication cost of one complete swap, in bytes, observed from the transmitted buffers over 10 swaps. Off-chain messages are counted, not only what reaches a ledger. **A range indicates a message whose size varied between runs, and only configuration 3's proof does so**: LaZer packs the proof's Gaussian response vectors with a variable-length code and then reports the length its encoder actually reached, so the packed size depends on the values sampled for that particular proof. Every other object here has a size fixed by its encoding, and the two ranged totals inherit that single varying term.

Message	Classical	LAS + Groth16	LAS + LaZer
Off-chain (exchanged directly between the parties)			
statement Y	33	4416	4416
proof π	—	128	30711–30760
$\hat{\sigma}_1$	162	6736	6736
tx_1	114	4497	4497
$\hat{\sigma}_2$	162	6736	6736
tx_2	114	4497	4497
off-chain total	**585**	**27010**	**57593–57642**
On-chain (published to the ledgers)			
$tx_1 + \sigma_1$ (chain 1)	178	11233	11233
$tx_2 + \sigma_2$ (chain 2)	178	11233	11233
on-chain total	**356**	**22466**	**22466**
total per swap	**941**	**49476**	**80059–80108**

elements whatever the circuit — but producing that proof means a multi-scalar multiplication over the whole constraint system, whereas LaZer's proof grows with the statement yet proves a lattice relation with lattice machinery, avoiding the encoding of a 23-bit modulus into a 254-bit pairing-friendly field. Verification runs the other way (13718.1 μs against 90720.8), succinctness paying off exactly where it is designed to.

Configuration 2's trade is worse than it looks. Pairing security is broken by Shor, so its proof is precisely the component a post-quantum adversary attacks: a post-quantum signature does not protect a swap whose fairness rests on a classical proof. Groth16 also needs a *trusted, per-circuit* setup (1.225142318s) whose secret must be destroyed for soundness, whereas LaZer's parameters are transparent. Configuration 3 is thus not merely the post-quantum option: it is faster end-to-end, needs no trusted party and removes a quantum vulnerability, paying only in bytes.

Communication is again the real price, with a usability consequence. A classical swap moves 941 B; the fully post-quantum swap moves 80059–80108 B, about 85.1× more, and puts 63.1× more on the ledgers — the multiple that matters economically, since a UTXO chain prices by size. A fully post-quantum swap needs 311097.1 μs and tens of kilobytes of off-chain messaging per exchange, overwhelmingly the proof, against 819.7 μs classically: comfortable on a laptop, but a wide enough gap that a mobile wallet's assumptions would not port naively. No constrained device was measured, so that point should not be over-read.

3.8 Transaction structure and what a UTXO chain would charge

The sizes above are the model ledger's own encoding (appendix A.4). Laid out instead in Bitcoin's wire format, what does a settled swap cost, and does it fit? Table 3.10 answers by projecting the *measured* object sizes onto the serialisation of [15, 13, 23]; the arithmetic is generated, and accepted only if it first reproduces two published reference figures.

The witness discount absorbs more than half of the post-quantum penalty. This rests on a transaction structure Bitcoin did not originally have. Before SegWit [15, 13] a signature travelled in the input script, where every byte was billed alike; the segregated witness bills witness bytes at one weight unit against four, and Taproot [23] supplies the output form used here. In raw bytes the post-quantum settlement is 58.9× the classical one; in virtual size — what fees are charged on — only 26.2×, a post-quantum signature being *entirely* witness data. That decision, taken long before anyone considered lattice signatures on Bitcoin, removes 55 % of the size penalty; on the original structure the same settlement would pay the undiscounted rate throughout. It is a concrete argument for UTXO chains as the venue, with no counterpart on the EVM, where calldata carries no comparable discount and native verification cost 56.6 M gas (table 3.7).

It fits the size limits, but not the verification rules. A settlement weighs 11,537 WU against the 400,000 WU standardness limit — 2.9 % of it — and the 4,000,000 WU block limit admits an upper bound of 346 settlements per block against 9,153 classical ones, ignoring competing transactions. A UTXO chain does

Table 3.10: One settled swap transaction in Bitcoin's wire format, field by field. The layout is identical in both columns (fig. 2.4); only the sizes differ. Object sizes are measured (run 20260730_162109); the field arithmetic is derived from [15, 13, 23] and self-checked against the published 110 vB P2WPKH reference spend. Virtual size is the quantity a UTXO chain prices, and the gap between it and total size is the witness discount. The model ledger's own encoding (table 3.9) totals 11233 B for the same objects, agreeing with the 11,255 B here to within 0.2 %: the two differ only in where the public key sits — inline in the model's output, in the witness under Bitcoin's layout — and in Bitcoin's marker, flag and compactSize prefixes.

Field	Classical (B)	LAS (B)
Base data — billed at 4 weight units per byte		
`version`	4	4
input count	1	1
input: outpoint + empty `scriptSig` + `sequence`	41	41
output count	1	1
output: value + `scriptPubKey`	31	43
`locktime`	4	4
base size	**82**	**94**
Witness data — billed at 1 weight unit per byte		
`marker` + `flag`	2	2
witness stack: signature σ + public key	107	11,159
witness size	**109**	**11,161**
total size	**191**	**11,255**
weight (WU)	437	11,537
virtual size (vB)	**110**	**2,885**

impose a limit, then, but a weight limit rather than a gas limit, and one swap sits well inside it — the contrast with the EVM, where the first native verifier exceeded the transaction gas cap outright and forced the change of venue.

Two obstacles survive, neither caused by the adaptor layer. No deployed consensus rule verifies a lattice signature: Script offers `OP_CHECKSIG` over ECDSA or BIP-340 Schnorr and nothing else, so configurations 2 and 3 could not settle on Bitcoin as it stands; [9] assumes only the venue, a UTXO chain "where the signature algorithm is replaced with a lattice-based signature scheme", and this study takes no position on which consensus route would deliver it. Separately, the objects collide with limits that exist regardless: the signature is 13.0× over the 520 B consensus stack-item limit and 84× over the 80 B standardness limit [3]. Both obstacles apply equally to an ordinary post-quantum payment: they are

properties of lattice signature sizes, not of adaptor signatures.

Both obstacles were put to a real client. A stock Bitcoin Core node carries the lattice objects — refused by default relay policy, yet consensus-valid and mined — but verifies nothing: an ordinary Schnorr signature authorises it. Adding one consensus rule, an `OP_SUCCESS` opcode [24] over the byte interface of section 2.3, gives a node mining a spend with no elliptic-curve signature, all 9 mutations tried being refused by it and accepted by an unmodified node — the control attributing the refusals to the new rule. The blocker is a consensus rule, not engineering. Its cost was measured: one verification runs 374 μs against the curve baseline it is compared with, a BIP340 Schnorr check's 33.0 μs — 11.3× — and a signature that fails late costs about as much as one that verifies. A patched node is not Bitcoin, so that conclusion stands; the exercise is functional, with no security argument (appendix A.13).

3.9 Threats to validity

Several factors qualify these results. All timings are machine-dependent, mitigated by reporting standard deviations, fixing one machine and compiler, emphasising ratios over absolutes, and by the cross-implementation check (section 3.3). The parameter set's concrete security level is not formally established, security proofs being out of scope, so every cross-scheme claim is qualified by table 2.1; the modulus is FIPS 204's $q \approx 2^{23}$ rather than the 2^{24} of [9], which leaves correctness unaffected ($q > 2\gamma$) but changes the security margin. The extracted witness is exact here, whereas the relaxed setting of [9] allows witness noise that can grow across long payment paths.

Two further factors are specific to section 3.7. The UTXO ledger is a model rather than a node (appendix A.4): real transaction objects, real serialised sizes and the real leak channel atomicity depends on, but no fees or confirmation latency, so on-chain byte counts are the honest proxy. That limit belongs to the venue rather than the study: configurations 2 and 3 need a consensus change, built and run in appendix A.13. Finally, the Stage-2 timings come from a single desktop machine (AMD Ryzen 7 7745HX with Radeon Graphics), so the usability discussion is an extrapolation, not a measurement on constrained hardware.

Chapter 4

Evaluation

This chapter judges the work against the objectives of section 1.3: the evaluation strategy and its alignment with those objectives, each objective against the evidence of chapter 3, then the implementation challenges that shaped the outcome and the limitations that bound it. Judgement of *what the artefact achieves* is made here; section 5.2 steps back to the process.

4.1 Evaluation strategy and its justification

The aim was to establish what adaptor functionality *costs*, so each measured difference had to be attributable to one cause. Four decisions follow, each tied to an objective.

Correctness is established before any cost is reported (objective 3). A fast but wrong scheme measures nothing, and no formal proof was in scope, so correctness rests on differential evidence: a pinned known-answer digest reproduced byte-for-byte by a second, independently written implementation (section 3.3). Internal self-consistency is not correctness — section 4.3 gives a case where every primitive passed in isolation and the assembled whole was still wrong.

The primary comparison is controlled, not merely convenient (objective 4). LAS is measured against its own simplified base at identical algorithm, parameters and primitives, so the difference between the two paths is the adaptor layer and nothing else. Comparing instead against the optimised CRYSTALS-Dilithium reference would confound the adaptor cost with parameter, packing and hint-vector differences; that comparison is therefore retained only as context, and the classical ECDSA adaptor only as a functionality-matched "price of

post-quantum" baseline.

Every measurement declares its boundary. Results are reported at two tiers (section 2.6), because an undeclared boundary is the commonest way a benchmark misleads — as it did twice here before the tiers were fixed. For the same reason each run gates on a directly counted rejection rate against its closed-form prediction, rather than inferring acceptance from timing.

Metrics follow the venue (objective 5). The application study reports execution time and communication bytes with off-chain messages counted, because the evaluated UTXO ledger has no gas metric — and because off-chain messaging is where the post-quantum cost lands.

4.2 Achievement of the objectives

Table 4.1 sets out the evidence behind each objective. All five were met; two carry qualifications that matter.

Objective 4's fairness claim rests on control rather than assertion: in Stage 1 the two compared paths share algorithm, parameters and primitives, and in Stage 2 the two LAS configurations share one public matrix and receive byte-identical inputs, verified at run time. The measurements are then defended from two independent directions — a closed-form gate on the rejection rate, and re-measurement under a different compiler and an independent statistical harness.

Objective 5 was met, but only after the first venue failed. On a local EVM a *complete*, validated native LAS verifier was measured at ≈56.6 million gas (table 3.7), above the EIP-7825 per-transaction cap, and an optimistic variant then made honest settlement cheap while leaving a worst-case fraud proof that also exceeds it. That negative result is quantified rather than assumed, and it motivated rebuilding the protocol over a UTXO ledger of the kind [9] assumes. That ceiling later proved to bound the *implementation* rather than the scheme: a claim transaction running a restructured verifier was mined inside the cap, at the target parameter set and message length (section 3.6). The venue decision therefore rests on cost, not impossibility.

Table 4.1: Each objective of section 1.3 against the evidence reported in chapter 3. All five were met; the evidence column states what each rests on.

Objective	Evidence
1. Literature and scheme selection	Adaptor signature selected as the exotic scheme of highest blockchain value; LAS [9] selected as the design with the shortest path from a standardised lattice signature (section 1.2).
2. Additive implementation	Lattice core reused byte-for-byte, zero upstream functions modified, made auditable by a two-branch diff; adaptor layer, wire encoding and application are new code (table A.7).
3. Validation	Functional, serialization, tamper and known-answer tests pass (section 3.1); an independent Rust implementation reproduces the wire format and the pinned digest byte-for-byte (section 3.3).
4. Fair benchmarking	Adaptor overhead isolated per operation at two declared tiers (table 3.2); two further baselines, computation separated from communication, rejection rate gated against theory, relative conclusions reproduced by the Rust port (table 3.4).
5. Atomic-swap demonstration	End-to-end two-party, two-chain swap settling both legs and asserting unspendability before adaptation (section 3.6); extended to a measured three-configuration study on a UTXO ledger (section 3.7).

4.3 Implementation challenges

Turning the simplified-Dilithium base into LAS was conceptually small — four added operations and one extra term in a hash — but the six problems of table A.1 consumed most of the engineering effort. Three themes recur. Two are failures that *pass every local test*: a loosened PreSign bound leaves the adaptor operations working while breaking ordinary verification two steps later, and a doubly-transformed public matrix produces a verifier whose parts agree with each other but not with the C implementation — both caught only by checking outside the component, which is why section 4.1 treats internal self-consistency as insufficient. Two more are measurement artefacts that would have biased the headline result had they survived, and are why every boundary is now declared and every run gated. The last is asymptotic rather than arithmetic, and generalises past this project: in constraint-system libraries the ergonomic interface and the

scalable interface are not the same interface.

4.4 Limitations

The validity threats of section 3.9 bound what the results *mean*. Two further limitations belong to the artefact rather than the measurement. The hint-free construction yields larger keys and signatures than optimised Dilithium — cryptographic optimisation given up for a transparent, testable artefact — and section 3.5 quantifies that price and shows it recoverable in principle, though only for the signature and not the statement. Configuration 2 instead gives up post-quantum assurance for a controlled comparison: it is the intermediate point that makes the proof-system comparison possible, not a stack anyone should deploy, since a post-quantum signature does not protect a swap whose fairness rests on a pairing assumption.

Chapter 5

Conclusion, critical reflection, and future work

5.1 Conclusions

The aim of section 1.3 was achieved within the stated prototype scope: the lattice core is reused unchanged, the adaptor layer, wire encoding and application are new tested code, the scheme passes functional, serialization, tamper and known-answer tests, an independent Rust implementation reproduces the pinned digest byte-for-byte, and an end-to-end two-party, two-chain swap runs.

The central question — what adaptor functionality costs over a plain signature, and what post-quantum operation costs over a classical one — is answered with measured data at declared boundaries. Adaptor functionality is almost free in computation: at most +8.3% at the core boundary, rising to +20.7–90.0% once the byte boundary is included, a rise that is serialization rather than arithmetic. The cost of LAS is therefore *communication* — the signature is 99.3% response vector, with the statement Y on top — and the classical comparison points the same way, the discrete-logarithm-equality proof making *its* adaptor layer proportionally the more expensive.

One design decision was tested rather than asserted. The simplified base was adopted because the adaptor appeared to need an exact verification identity; building the same construction on unmodified ML-DSA showed that belief was half wrong (section 3.5). PreSign and PreVerify must indeed be new algorithms — the naive port fails outright — but Verify need not be, and a standard FIPS 204 verifier accepts the adapted signature. That route would halve the signature

at equal computation, yet leave the statement Y untouched, so it relocates the communication bottleneck rather than removing it.

Taking the protocol to a UTXO ledger — the venue [9] assumes — sharpens that conclusion, because at the application level the signature is not the dominant cost at all: the role-A proof consumes 99.3% of a swap under Groth16 and 98.6% under LaZer. Changing only the proof system, the *post-quantum* prover is 53.1% faster end-to-end while being 61.9% larger. The fully post-quantum configuration is thus at once the faster one, the one with the weaker trust assumption, and the only one Shor does not break, paying for all three in bytes.

All five objectives were met (table 4.1). The principal qualification is scope: no concrete security is formally analysed, and both applications run on modelled ledgers rather than live networks.

5.2 Critical reflection

5.2.1 What was achieved

The central strategic choice — to build *additively* on a standardised reference rather than reimplement lattice arithmetic — paid off twice. It made the implementation tractable, and it produced an unusually strong provenance argument: the security-critical arithmetic is byte-for-byte the published CRYSTALS-Dilithium reference, with a two-branch diff that makes the claim auditable rather than assertable. Being language-agnostic, the strategy also made the second implementation cheap, upgrading the correctness argument from one tested codebase to two agreeing byte-for-byte, and exposing every timing conclusion to an independent harness. If one methodological result deserves to outlive this dissertation, it is that pairing a pinned known-answer digest with an independent reimplementation is a cheap and decisive way to validate a cryptographic port.

Two further things went better than expected. Confronting the fairness problem honestly — comparing LAS against a parameter-matched base rather than optimised Dilithium — converted an apparent size deficit into a precisely attributed one. And counting rejection attempts rather than inferring them from a timing ratio corrected a materially wrong earlier figure, a lesson now institutionalised as a gate every run must pass.

A third result belongs here because it reads as failure and is not. Three

optimisations this dissertation could have left as speculation — compressing the statement inside the construction, amortising the proof across a batch, and proving the relation with a succinct post-quantum system — were implemented and measured instead, and all three returned negative answers (section 5.3). An untested future-work suggestion costs nothing to write and tells a reader nothing; each of these instead names the mechanism that defeats it and closes the direction, which is worth more than the optimisation would have been.

5.2.2 What fell short

Three shortfalls are worth stating plainly, over and above the scoped limitations of section 4.4.

First, **on-chain settlement was measured, then solved narrowly, and still not deployed**. The first Solidity verifier sat far above the cap, and the optimistic variant only moved the problem into a fraud proof that also exceeded it. The dissertation then drew the wrong lesson — recording the cause as structural, SHAKE256 not being EVM-native, and inferring that local optimisation could not close the gap. Re-expressing the same predicate closed it (section 3.6): an inference asserted rather than measured, which cost the project that question for as long as it stood. What remains unsolved is cost — settlement far dearer than a classical claim, fitting by a thin margin at one parameter set and one message length, on no live network.

Second, the applications are **exploratory demonstrators, not products**. Both settle over modelled ledgers and assert the properties that matter, but neither runs against a live network. For the UTXO study this is partly forced: Bitcoin Script cannot verify a lattice signature, so the post-quantum configurations could not settle on a real node without a consensus change — exactly the assumption [9] makes. The reported costs are therefore an implementation-level feasibility estimate rather than a deployment figure, and since the omitted overheads add cost, not an upper bound on one.

Third, the concrete security of the chosen parameter set is **unanalysed by design**. This was a sanctioned scope decision, not an oversight, but it bounds what the performance numbers mean: they characterise a faithful, testable artefact at a stated engineering setting, not a parameter set anyone should deploy.

5.2.3 What would be done differently

The clearest change is **target selection for the application**. Substantial effort went into taking the verifier on-chain in Solidity before the gas cost was known to be prohibitive. The framing was wrong as well as the venue: the project asked "can a post-quantum signature be verified on chain?" when what mattered was "what dominates a post-quantum swap?" — and the answer never touches a chain at all. Beginning on the UTXO side would have reached that sooner, with the EVM measurement retained for what it does establish: why the venue was changed.

Second, **measurement discipline would come first**: both early artefacts — the biased acceptance estimate and the asymmetric timed loops — followed from building the benchmark before deciding what boundary it measured. Third, in the same spirit, the ML-DSA experiment should have come *before* the simplified scheme was justified in writing: a design decision defended by argument alone is one nobody has tested, and testing it took days, not weeks.

5.3 Future work

- **A statement that is small by design.** With the signature halved and Y untouched (section 3.5), the statement is the dominant object. Compressing Y *inside* this construction was tested and fails: truncation is invisible to the adaptor's own functions, which hash whatever statement they are handed, and fatal at both boundaries that matter: ordinary verification recomputes $A\mathbf{z} - c\,t$ against the true Y, and EXTRACT accepts only on the exact relation. Deriving Y from its seed compresses it completely and hands over the witness. What is open is not a better encoding but a different hard relation whose statement is small by construction.

- **Reduce the proof.** Both routes proposed here were measured, and both fail at this scale. Batching amortises the wrong cost — the Groth16 proof is constant in the batch and never the bottleneck, while the LaZer proof shrinks per swap only by making proving and verification several times worse. A *succinct* post-quantum system that is also zero-knowledge, which [9] requires of π, run under LaBRADOR, loses to the deployed prover on every axis — on size by that library's own estimate rather than a packed

proof, on time outright: succinctness is asymptotic, and one instance of this relation is too small to reach it. What is open is a construction whose proof is small at *this* scale.

- **Reduce the cost of one-transaction verification, and carry it across parameter sets.** Fitting under the cap was proposed here as future work and then answered, without any of the routes originally predicted (section 3.6). Two narrower questions remain. The margin is a message-length budget, tying the result to the parameter set and message length measured, and the verifier's dimensions are fixed at build time, so no other set was evaluated. And fitting is not the same as being worth doing: settlement stays far above a classical claim, and the hash is the dominant measured term, so that is where a substantial further reduction would have to come from — though what native support for it would cost, and whether the result would be competitive, was not evaluated here. A precompile for ML-DSA verification is specified as a draft, covering only NIST level II [7], and was declined from the next scheduled upgrade [2]; it would not verify LAS, bearing instead on the ML-DSA route of section 3.5, whose adapted signatures a stock FIPS 204 verifier accepts.

- **Analyse the security of the ML-DSA variant.** Section 3.5 establishes that committing to HighBits$(w + Y)$ is *functionally* sound; whether it preserves ML-DSA's unforgeability argument is unexamined, and is the prerequisite before that route could be recommended over the simplified scheme.

- **Take the UTXO study to a live network.** *Signet* would supply the classical configuration with the fees, propagation and confirmation latency the model cannot. For configurations 2 and 3 no stock Bitcoin test network can supply a deployment figure: the blocker is a consensus rule, not engineering. Appendix A.13 adds that rule experimentally and settles a lattice-authorised spend; Bitcoin as deployed cannot, so that figure awaits adoption.

Appendix A

Code listings and reproduction

Per the writing guide, code snippets belong in an appendix rather than the body. Only the most representative listings are included here; the full source is in the project repository.

A.1 Building and reproducing the benchmarks

The evaluation is reproducible from the commands below. The upstream Dilithium reference is pinned at the repository's initial commit (`2374d22`); the classical baseline library is the BlockstreamResearch `secp256k1-zkp` `ecdsa_adaptor` module at commit `95b9835`; the Rust port builds on a vendored copy of the `fips204` crate (commit `c948882`). The C toolchain is the system compiler (`cc`, Ubuntu GCC 13.3.0) with `-O3`; the Rust toolchain is stable `rustc` (the committed run used 1.96.0) in release profile.

Listing A.1: Fair benchmark and tests (first-stage evaluation). The suite script runs everything below, saves the logs and parsed tables as a timestamped evidence run, and regenerates every figure, table, and inline number of this report from that run.

```
# the one supported evidence pipeline (all tests + all benchmarks +
    report sync)
scripts/run_benchmark_suite.sh

# or individually:
cd ref
```

```
# fair, parameter-matched base-vs-adaptor benchmark at each parameter
    set
make test/bench_levels_paper && ./test/bench_levels_paper
make test/bench_levels2 && ./test/bench_levels2
make test/bench_levels3 && ./test/bench_levels3 # target setting
make test/bench_levels5 && ./test/bench_levels5
# correctness, serialization/tamper, known-answer tests
make test/test_las3 && ./test/test_las3
make test/test_serde3 && ./test/test_serde3
make test/test_kat3 && ./test/test_kat3
# end-to-end atomic swap incl. the proof of knowledge pi
# (needs the one-time LaZer build below)
make test/test_zkp3 && ./test/test_zkp3
make test/test_swap3 && ./test/test_swap3
```

Listing A.2: Classical adaptor baseline (one-time clone).

```
git clone --depth 1 https://github.com/BlockstreamResearch/secp256k1-
    zkp \
    third_party/secp256k1-zkp # tested at commit 95b9835
cd ref && make test/bench_classical && ./test/bench_classical
```

Listing A.3: Proof-of-knowledge library (one-time clone and build; see the repository README for the dependency recipe).

```
git clone https://github.com/lazer-crypto/lazer third_party/lazer
cd third_party/lazer && make lazer.h liblazer.a
# the generated proof parameters (ref/relation_zk_params.h) are
    committed,
# so SageMath is needed only to REgenerate them, never to build
```

The Rust port is reproduced with the standard Cargo tooling; the test gate runs first because the benchmarks refuse to time unverified code, and the known-answer test asserts the same pinned digest as the C implementation (`b4a10ffb...03be`).

Listing A.4: Rust port: test gate, statistical benchmark, protocol driver, and size report (one suite script, or the four steps individually).

```
# the one supported Rust evidence pipeline (tests + benchmarks + report
    sync)
scripts/run_rust_bench_suite.sh
```

```
# or individually:
cd rust/fips204-las
cargo test --lib --tests # gate: includes the pinned KAT digest
cargo bench --bench las_bench # Criterion.rs statistical harness
cargo run --release --example bench_levels # protocol driver (mirrors
    the C driver)
cargo run --release --example size_report # packed component sizes
```

A.2 The timing-benchmark procedure

The pseudocode below is the procedure both protocol drivers implement (`ref/test/bench_levels.c` and the Rust `examples/bench_levels.rs`, deliberately line-for-line mirrors), backing the methodology of section 2.6. Ordering matters: correctness is asserted *before* timing on the very objects about to be timed, so no failure path is ever measured, and the rejection gate runs over the *timed* calls themselves, so the gate certifies the same data the tables report. The same procedure runs once per parameter set.

Listing A.5: The timing-benchmark procedure (both tiers). $R = 5$ repetitions; $N = 500$ (sign-class) or 1000 (verify-class) iterations; the rejection gate follows eq. (2.2).

```
Input: parameter set (n, l, kappa); fixed pp seed; fixed 33-byte
   message
 1 Setup: expand A from the fixed seed; KeyGen; relation Gen (Y, y)
 2 Gate 0 (correctness): run the full adaptor contract on the exact
   objects about to be timed (PreVerify accepts; basic Verify rejects
   the pre-signature; Adapt verifies; Extract returns y exactly);
   abort the run on any failure
 3 for each operation op in {KeyGen, Sign, Verify, PreSign, PreVerify,
   Adapt, Extract}: # core tier
 4 for r in 1..R:
 5 a0 <- attempt counter # sign-class only
 6 t0 <- monotonic clock
 7 repeat N times: run op (structures in / structures out;
            fresh masking randomness inside every sign-class call)
 8 run_r <- (clock - t0) / N
```

```
 9 att_r <- (attempt counter - a0) # sign-class only
10 report mean +/- sample SD over run_1..run_R
11 Gate 1 (run validity): for Sign and for PreSign, the mean attempts
    per call over ALL R*N timed calls must satisfy
        |mean - E| <= 5 * E*sqrt(1-1/E) / sqrt(R*N)
    where E = 1/p_acc is the closed-form expectation; abort on failure
12 repeat steps 3-11 at the packed boundary (packed bytes in /
    packed bytes out, validating decode + encode inside the call)
13 emit packed sizes (communication is fixed: measured once, no SD)
```

The Criterion.rs harness measures the same operations under its own protocol (3 s warm-up, 300 samples per operation, outlier classification, bootstrap confidence intervals) and applies the same Gate 1; it shares no timing code with the drivers, which is what makes its agreement an independent check rather than a re-run.

A.3 Methodology details

This section carries the derivations and experimental-design detail deferred from chapter 2, so that the body states the decisions and this appendix justifies them in full.

Why the per-attempt acceptance probability is secret-independent. Each coefficient of the mask is uniform on the $2\gamma + 1$ values of $[-\gamma, \gamma]$, and the secret-dependent term satisfies $\|c\,r\|_\infty \le \kappa$, because the challenge has exactly κ nonzero coefficients of ± 1 and the secret is ternary. Each response coefficient $z_i = y_i + (c\,r)_i$ is therefore uniform on a window of $2\gamma+1$ consecutive integers whose centre is displaced from zero by at most κ. The acceptance interval $[-(\gamma-\kappa),\ \gamma-\kappa]$ is narrower than that window by exactly κ on each side, so it lies wholly inside the window *wherever* the displacement falls. Each coefficient is thus accepted with probability exactly $\big(2(\gamma-\kappa)+1\big)/(2\gamma+1)$, independently of the secret — which is precisely why rejection sampling makes the published response simulatable, and hence why it leaks nothing about r. Raising this to the $(n+\ell)\,d$ coefficients of one attempt gives eq. (2.2).

The run-validity gate in full. Over a run's timed sign-class calls the measured mean attempts $\bar{A}$ must satisfy $|\bar{A} - E| \le 5\,\sigma/\sqrt{n_{\text{calls}}}$, where $E = 1/p_{\text{acc}}$ is the

closed-form expectation of eq. (2.2) and $\sigma = E\sqrt{1 - 1/E}$ is the geometric standard deviation. The test is applied separately for Sign and for PreSign, at both timing tiers, in C and in Rust, and a run that fails it aborts instead of producing evidence. The resulting band is $\pm\sim 8\%$ at the C drivers' 2500 timed calls and $\pm\sim 1\%$ at the Criterion harness's $\approx 10^5$, which is why only the latter resolves the 2% theoretical separation between Sign and PreSign. The gate is a consistency check on the observed rejection rate, not a proof of sampler correctness: it provides evidence that the rejection behaviour is consistent with theory and that rejection-sampling variation is sufficiently controlled for the reported mean.

The Groth16 circuit. No pairing-based proof of this lattice statement was available to reuse, so the circuit is new, and it is far cheaper than the statement suggests. The map $r' \mapsto Ar'$ is linear with public coefficients, so each row is an affine combination of witness variables and needs no witness–witness multiplication — the expensive ingredient in a rank-one constraint system. The public linear map is still enforced, through linear constraints; the only non-linear and range constraints are the ternary bound, enforced as $r'^3 = r'$ per coefficient, and the reduction modulo q, enforced with a range-checked quotient per row. The constraint count is therefore linear in the parameter set's dimensions rather than in their product, and the same statement would have been intractable had it involved products of two *secret* polynomials. Because the matrix is held in the number-theoretic-transform domain and is exported by neither implementation, it is recovered by evaluating the scheme's own linear map on unit vectors, so the circuit encodes the map the signature scheme actually computes with rather than a re-derivation that could drift from it.

Measurement invariants of the UTXO study. Two invariants are enforced rather than assumed. A phase must be timed in *every* run or in none, so that no mean is silently taken over the subset of swaps that reached it; and communication subtotals are computed within each run before being reduced across runs, since summing each message's minimum would describe a swap that never occurred. One-time costs — expanding A, building the elliptic-curve context, and Groth16's trusted setup — are performed before timing begins and reported separately, being amortised over every swap rather than paid per swap. The rejection gate applies here too, but on a dedicated batch of 2000 pre-signature calls rather than

on the two per swap the protocol itself performs: over so few calls the five-sigma band is wider than the distance from the expected attempt rate to a loop that never rejects at all, so a gate on that sample would have been vacuous.

Matching the boundaries of the classical comparison. The two libraries of table 3.6 do not draw their measurement boundaries in the same place, so the comparison states both. The classical column is the library's *hybrid* native-API boundary: PreSign, PreVerify, Adapt and Extract pack or parse the 162-byte pre-signature *inside* the timed call (packed-like), whereas KeyGen, Sign and Verify exchange in-memory structs with their wire codecs outside it (core-like). LAS is therefore shown at both its core and its packed tier, measured on the C implementation, and the tier-matched column pairs each operation with whichever LAS tier corresponds to the classical boundary for that operation — the core tier for KeyGen/Sign/Verify and the packed tier for the four adaptor operations. Timings are the mean ± sample standard deviation over 10 runs × 1000 iterations classically and 5 repetitions × 500/1000 iterations for LAS, on the same machine. Comparing every row against a single LAS tier instead would count the wire codec on one side only, for four of the seven operations, which is why the matching is done per operation rather than per table. Two object-level differences lie behind the size rows: LAS KeyGen also produces the statement and witness, which take the public-key and secret-key size, and the classical pre-signature is a syntactically distinct object — an ECDSA signature plus a discrete-logarithm-equality proof — whereas the LAS pre-signature shares the signature format exactly.

What the restructured on-chain verifier changes, and how it was checked. The two `claimLASVerified*` rows of table 3.7 decide the same predicate as `base_verify`: decode, norm bound, $w' = A\mathbf{z} - c\,t$, and the SHAKE256 challenge check. The first mirrors those steps directly from the vendored ZKNox primitives and is validated against the C reference. The second re-expresses them: the public key is registered in the number-theoretic-transform domain and $c\,t$ folded before a single inverse transform, parameters are read from calldata rather than decoded into memory, and the sponge avoids the per-byte absorb. It reproduces the first verifier's packed w' byte-for-byte on the C golden vectors, and agrees with it on accept and reject for the golden signature and for a tampered response, challenge and message. The registration condition that qualifies this equivalence is stated with the table itself.

One deliberate exception to reproducibility. Every other input derives from the pinned seed, but Groth16's trusted setup and each of its proofs draw fresh operating-system entropy. Both are security requirements rather than preferences: a reproducible setup would leave the toxic waste recoverable and void soundness, and reusing proof randomness across statements breaks zero-knowledge. The measurement consequence is stated rather than hidden — proof *size* is unaffected, since a Groth16 proof is three group elements regardless, but proving *time* includes the entropy draw and so carries a little extra run-to-run variance.

A.4 The modelled UTXO ledger

The ledger behind section 2.7 is a model, not a node: it has no peer-to-peer layer, no mempool policy, no fee market, no reorganisations and no script interpreter. What it does provide is what the study needs — real transaction objects with canonical serialisation, a signature hash computed over the transaction rather than over an abstract message, and the public observability on which the protocol's atomicity depends, since a settled transaction's signature can be read back off the ledger by anyone, which is exactly how the witness leaks. An output carries an amount plus a spending condition: a signature check under a public key, with an optional timelocked refund branch.

Deploying instead on a real Bitcoin node is not a matter of effort. Bitcoin Script cannot verify a lattice signature, so configurations 2 and 3 could not settle there without a consensus change — which is precisely the assumption [9] makes when it states its applications over a UTXO-based chain *whose signature algorithm is replaced with a lattice-based signature scheme*, and the reason that sentence is stated as an assumption rather than left implicit.

A.5 Wire encoding and the validating decoder

The packed field widths behind section 2.3 are 23 bits per coefficient for the public key and statement, 2 bits for the ternary secret and witness, and 18 or 19 bits for the response z depending on the parameter set, since that field must hold $2(\gamma - \kappa)$ and γ grows with $\kappa\, d\, (n + \ell)$. Because a verifier cannot trust its input, the decoder is defensive: it rejects a public-key coefficient $\geq q$, the invalid ternary code, and any response field outside the parameter set's band, while the encoder

symmetrically rejects a non-ternary secret or a response exceeding $\gamma - \kappa$. This is the interface a Solidity, precompile or zero-knowledge-circuit verifier would expose, and it is the object the tamper test of section 3.1 attacks.

One encoding decision is worth recording. Following FIPS 204 [18], the wire format carries not the challenge polynomial c but only the digest $\tilde{c}$ from which it is derived, every consumer re-deriving $c = \mathsf{SampleInBall}(\tilde{c})$ after decoding. This is a computation-versus-storage trade: storing the expanded polynomial would save the re-derivation at every decode, while storing the digest keeps the signature smaller.

The digest *width* is an implementation choice rather than a requirement of [9], which specifies H only as a random oracle onto the challenge set C and fixes no encoding. It is therefore taken from FIPS 204, whose $\mathsf{SampleInBall}$ consumes a seed of $\lambda/4$ bytes, with $\lambda = 128, 192, 256$ for ML-DSA-44, -65 and -87 giving 32, 48 and 64 bytes. The width tracks the LAS parameter set and not the reference build mode, since the sweep compiles several LAS sets against a mode chosen only to satisfy the reference's parameter header: the ML-DSA-65-aligned target set takes 48 bytes, the ML-DSA-44- and ML-DSA-87-aligned LAS sets take 32 and 64, and the historical paper-parameter reference set, which corresponds to no ML-DSA parameter set, takes 32 by this project's choice. Matching these reusable parameters aligns the primitives and the challenge-digest strength only — LAS retains its own ternary secret distribution, rejection bounds, exact hint-free relation and adaptor algorithms — so it is not a claim that the scheme is ML-DSA-65 or inherits its security category.

A.6 The atomic-swap demonstration and its proof of knowledge

The narrated demonstration referenced in section 3.6 follows the swap protocol — Figure 1 of [9] — *verbatim*. Two parties swap coins across two ledgers with no trusted third party and no on-chain scripts. Alice, the witness holder, commits first: she draws a fresh statement/witness pair (Y, y), produces a zero-knowledge proof π that Y has a *ternary* preimage $(\exists\, r' : \mathbf{A}r' = Y \wedge \|r'\|_\infty \leq 1)$, pre-signs the transaction spending her own coin, and sends $\{Y, \pi, \hat{\sigma}_1\}$ in one off-chain message. Bob pre-signs his own transaction against the *same* Y only after both π and Alice's pre-signature verify — the protocol's abort gate; at that point neither

pre-signature is spendable. Alice, who knows y, adapts and publishes her signature to claim Bob's coin; the act of publishing reveals y, which Bob extracts and uses to adapt and publish his own signature. Either both legs settle or neither does. The demonstration prints this narrative and asserts every step, including the counterfactuals that a pre-adaptation signature fails basic verification and that π is rejected against any other statement.

The proof π reuses the LaZer lattice zero-knowledge library [14] rather than a proof system written from scratch — the same reuse posture applied throughout. Because LaZer proves per-partition binary coefficients or exact Euclidean norms, the ternary statement is encoded by binary decomposition, $[\mathbf{A} \mid -\mathbf{A}]\,(r_+ \,\|\, r_-) = Y$ with both halves proven binary, which is equivalent to knowledge of a ternary preimage $r' = r_+ - r_-$. The generated parameter set reports a knowledge error below 2^{-127} under Module-SIS and a proof size of 32 046 B — the parameter set's own figure, larger than the proof bytes observed on the wire in table 3.9 and not to be conflated with them — with proving and verifying each completing well under a second. A dedicated test suite asserts completeness, rejection of every sampled single-byte tamper, rejection against a wrong statement, and that the prover refuses a non-ternary witness. A small ledger abstraction — accounts with balances, a block height, and adaptor-locked contracts, the scriptless analogue of a hash-time-locked contract with the hash lock replaced by the statement Y and the time lock by a timeout height — additionally carries two asserted scenarios: the cross-chain happy path, and a timeout/refund in which a premature refund is rejected and both parties recover their funds.

A.7 The optimistic (Naysayer) on-chain verifier

Because the first native Solidity verifier exceeds the per-transaction gas cap (section 3.6), an *optimistic* alternative was also implemented and measured: `LASNaysayer`, a poqeth-inspired Naysayer verifier [12] for the lattice setting — poqeth itself implements only hash-based and multivariate schemes and rules Dilithium out on public-key size. Instead of re-executing verification, the contract accepts a settlement optimistically and lets any party dispute a single failing step within a challenge window. Verification is decomposed into three locally checkable invariants — the response norm, each recomputed commitment coefficient (with the challenge derived on chain), and the final hash — so a dispute recomputes only

one coefficient by schoolbook convolution and never runs the number-theoretic-transform batch. An adversarial suite of nine tests passes, covering soundness (no dispute voids a valid settlement), completeness for each fault type (including a C-exported pure hash-fault vector against which only the hash dispute succeeds), the challenge-window and replay guards, input validation, and pull-payment isolation against a hostile recipient.

The result is deliberately reported as mixed. Honest settlement is cheap — an optimistic claim costs approximately 1.1 million gas and finalisation approximately 27 000, since the batch never executes on the happy path — but the binding cost is the worst-case fraud proof. The norm dispute costs approximately 34 000 gas; the commitment dispute, in a baseline that passes the whole public matrix in calldata, approximately 13.9 million gas of execution together with an approximately 246-kilobyte calldata payload; and the hash dispute approximately 28.2 million gas. These are execution-gas figures, so the full per-transaction totals — adding the fixed intrinsic cost and the calldata — are strictly larger. The hash dispute alone therefore exceeds the EIP-7825 cap of 16 777 216 by roughly 1.75× on execution gas before any calldata, because it must run a complete SHAKE256 in Solidity, which is not an EVM-native hash — unlike the Keccak-based schemes of [12]. The optimistic mode thus moves cost off the honest path but does *not*, in this baseline, bring the worst-case dispute within a single transaction; doing so would require opening only the single disputed matrix row through a Merkle commitment rather than sending the whole matrix, and a hash dispute that avoids a full SHAKE256 pass.

A.8 The six implementation challenges

The themes drawn from these six challenges are discussed in section 4.3; the table itself is placed here because it runs to more than a page.

Table A.1: The six challenges that shaped the implementation. A seventh, cross-cutting difficulty — reproducing the C scheme byte-for-byte in Rust, down to how each language's uniform sampler discards rejected blocks — is discussed with the cross-validation results (section 3.3); the pinned digest is what turned it into a mechanical process. One caveat belongs with the gas figure of challenge 5: the reused ZKNox primitives come from a self-described non-production library, so the measurement is the honest cost of *a* numerically-correct verifier, not an endorsement of that library.

Challenge	The problem	Resolution and lesson
1. The rejection-bound budget	PreSign must reject at $\gamma - \kappa - 1$, one tighter than eq. (2.1). At $\gamma - \kappa$ instead, PreSign, PreVerify and Extract all still succeed, but the *adapted* signature can exceed the bound and be rejected by ordinary Verify — silently, probabilistically, two operations from the cause.	Prevented by reasoning through the norm budget ($\|y\|_\infty \leq 1$, hence $\|\hat{\mathbf{z}} + y\|_\infty \leq \gamma - \kappa$) rather than found by testing.
2. Reference optimisations appear to fight the adaptor identity	Power2Round, the hint vector and high/low-bit decomposition discard low-order information, which appeared to rule out the exact identity $A\mathbf{z} - c\,t = w + Y$; disabling them was treated as a design requirement.	Establishing *which* to disable was a prerequisite to a working PreVerify. The belief that they *had* to be was later tested and proved too strong: with the whole commitment path moved onto $w + Y$ the adaptor works with all three enabled (section 3.5): a sufficient route, not a necessary one.
3. Living in another scheme's namespace	The reference globally defines single-letter parameter macros — its `N` is the ring degree, its `K` the module rank — while LAS needs different dimensions with the same natural names, so a constant silently inherits the wrong value through the preprocessor instead of failing to compile.	Forced prefixed parameters and a documented C–Rust–paper name map.

continued on the next page

Table A.1 — continued from the previous page

Challenge	The problem	Resolution and lesson
4. Benchmarking a variable-time scheme fairly	Rejection sampling makes sign-class timings heavy-tailed. An acceptance rate inferred from a timing ratio was biased (≈23% where direct counting gives 38.2%), and the two languages' base signing paths initially used different norm-check strategies *inside* the timed loop — invisible to the known-answer test, but a work-profile asymmetry biasing the comparison.	Attempts are now counted and gated against theory, the drivers mirror each other line-for-line, and every boundary is declared (section 2.6).
5. An error that was self-consistent	Porting the verifier to Solidity, the public matrix A' — stored by the reference *already in the NTT domain* — was transformed a second time. The recomputed commitment and challenge agreed *with each other*, so every primitive passed in isolation while the whole disagreed with the C implementation.	Found only by recomputing the challenge digest against the reference's; thereafter the port was validated in stages against exported vectors.
6. A constraint system that was asymptotically wrong	The Groth16 circuit first accumulated its dense rows with ordinary field-element arithmetic, cloning the accumulated linear combination on each addition and so costing work quadratic in the row width. The symptom was an apparent hang, the work happening during setup before the driver prints.	Rebuilding each row once as an explicit linear combination against the low-level constraint-system interface made the cost linear; setup now completes in 1.225142318s.

A.9 Test types and parameter notation

A.10 Full benchmark data tables

The tables in chapter 3 carry the primary results; this section gives the supporting data at every parameter set, produced by the commands above (listing A.1). Table A.5 lists every core-tier per-operation timing with its standard deviation

Table A.2: The four test types, what each asserts, and what it would catch. The functional suite's statement-binding clause — that a pre-signature *fails* basic verify — is the tripwire that distinguishes an adaptor signature from a signature; the known-answer digest is what the second implementation is checked against (section 2.5).

Test type	Scale and assertions	Catches
Functional	1000 iterations per Dilithium mode (2, 3, 5), fresh parameters, keys and message each time; asserts the full adaptor cycle — pre-signature pre-verifies, *fails* basic verify, adapts to a valid signature, and extraction returns the witness exactly — plus a sign/verify round trip and one-bit-forgery rejection	any break in the adaptor contract; statement binding
Serialization	256 random instances; exact round-trip for keys and every signature variant	encoder/decoder asymmetry
Tamper	every one of the 6736 bytes of a valid packed signature flipped in turn; all must break verification; malformed encodings must be rejected	a decoder that trusts its input
Known-answer	all inputs fixed, deterministic API, packed bytes of every object folded into one SHAKE256 digest compared against a pinned 32-byte value	any unintended change to sampling, algebra or encoding, on any machine

(the data plotted in fig. 3.1, and the source of the core block of table 3.2); table A.6 gives the component byte sizes at every parameter set (the target setting is the body table table 3.5).

A.11 Auditing the reuse claim

Table A.7 gives the per-file classification behind the "reused vs. modified vs. added" claim of section 2.2. The claim was established by keeping the pristine upstream tree on its own branch and diffing it against the project branch: the commands below print nothing for the lattice core, which is what makes "zero upstream functions modified" an auditable statement rather than an assertion.

Table A.3: Security and scheme parameters: the symbol, its meaning, and the value(s) it takes across the parameter sets, *listed in the row order of* table 2.1 (LAS-2020/845 reference; Simplified Dilithium-II; -III; -V). The notation follows the LAS paper [9]: the ring degree is its d, and this build runs at $d = 256$, the ring degree of the reused Dilithium NTT. The one exception is the challenge-digest width $|\tilde{c}|$, which [9] does not fix — it specifies H only as a random oracle onto the challenge set C — so the width is an implementation choice, taken from FIPS 204 wherever a set aligns with an ML-DSA one (appendix A.5). The figure and table captions in chapter 3 refer back to these names and values.

Symbol	Meaning	Value(s): LAS-2020/845 reference, Simplified Dilithium-II/III/V
n	module rank: rows of A, length of public key t	4, 4, 6, 8
ℓ	extra columns of A' (secret-vector extension)	4, 4, 5, 7
$M = n + \ell$	response dimension (number of polynomials in $z, \hat{z}, y$)	8, 8, 11, 15
κ	challenge Hamming weight (number of ± 1 in c; Dilithium τ)	60, 39, 49, 60
γ	rejection / masking bound, $\gamma = \kappa\, d\, (n + \ell)$	122 880, 79 872, 137 984, 230 400
d	ring degree $(X^d + 1)$	256 (all sets)
q	modulus from the reused NTT, $q \approx 2^{23}$	8 380 417 (all sets)
$\lvert\tilde{c}\rvert$	challenge-digest width in bytes: FIPS 204's $\lambda/4$ at the three ML-DSA-aligned sets; a project choice at the LAS-2020/845 reference set, which that rule does not cover	32 (project choice), 32, 48, 64

Listing A.6: Branch diff establishing that the lattice core is untouched (no output means identical).

```
git diff --name-status dilithium-baseline main -- ref/
git diff dilithium-baseline main -- ref/poly.c ref/ntt.c ref/sign.c \
    ref/packing.c ref/polyvec.c ref/reduce.c ref/rounding.c ref/params.h
```

A.12 Selected code listing: Adapt and Extract

The adaptor mechanism reduces to two short routines. ADAPT adds the witness to the pre-signature's response; EXTRACT recovers the witness by subtraction

Table A.4: The measurement boundaries. Comparisons are only ever made within one boundary. The classical column is not a uniform tier but a *hybrid*, because `secp256k1-zkp` exposes the pre-signature only in packed form: the four adaptor operations behave like the packed tier and the three base operations like the core tier.

Boundary	What crosses it	What it answers
Core tier	in-memory structures in, structures out	what the adaptor *algebra* costs, isolated from encoding
Packed tier	packed wire bytes in and out, including the validating decode of every input and encode of every output (section 2.3)	what a wire or on-chain consumer pays per call with nothing cached; one operation's byte boundary, *not* a whole-protocol cost
Hybrid native API (classical only)	the boundary `secp256k1-zkp` itself exposes: PreSign, PreVerify, Adapt and Extract pack or parse the 162-byte pre-signature *inside* the timed call; KeyGen, Sign and Verify exchange parsed structs with the wire codecs outside it	the classical cost as the unmodified library actually charges it

and confirms it against the statement. Both reuse the reference's polynomial arithmetic verbatim.

Listing A.7: `las_adapt` and `las_ext` from `ref/las.c`.

```
int las_adapt(las_sig *sig, const las_sig *presig, const uint8_t *m,
   size_t mlen,
             const las_pk *Y, const las_sk *y, const las_pk *pk, const
                 las_pp *pp) {
  unsigned int j;
  if(las_preverify(presig, m, mlen, Y, pk, pp))
    return -1;
  sig->c = presig->c;
  for(j = 0; j < LAS_M; ++j) { /* z = z_hat + y_witness */
    poly_add(&sig->z[j], &presig->z[j], &y->s[j]);
    poly_reduce(&sig->z[j]);
  }
  return 0;
```

Table A.5: Per-operation timing, reported independently (μs per operation; mean $\pm$ sample standard deviation over 5 repetitions $\times$ 500 sign-class / 1000 verify-class iterations; `bench_levels`, machine of record in section 2.6). KeyGen, Sign, and Verify *are* the basic signature and are reused unchanged by LAS; PreSign, PreVerify, Adapt, and Extract are the four adaptor operations. Public-parameter Setup, a one-time global cost, runs in 25–88 μs and is omitted. Parameter sets and symbols: tables 2.1 and A.3.

Operation	LAS-2020/845 reference $(4, 4, 60)$	Simplified Dilithium-II $(4, 4, 39)$	Simplified Dilithium-III $(6, 5, 49)$	Simplified Dilithium-V $(8, 7, 60)$
Basic signature (reused unchanged by LAS)				
KeyGen	32.3 ± 0.2	34.1 ± 0.1	50.9 ± 0.9	69.0 ± 0.1
Sign	257 ± 9	314 ± 14	489 ± 17	518 ± 15
Verify	70.0 ± 2.1	74.1 ± 0.7	111 ± 3	139 ± 0.37
LAS adaptor operations				
PreSign	265 ± 8	328 ± 19	522 ± 32	540 ± 22
PreVerify	71.6 ± 0.2	75.2 ± 0.6	115 ± 2	146 ± 2
Adapt	73.7 ± 0.3	77.8 ± 0.2	118 ± 2	151 ± 2
Extract	26.0 ± 0.1	28.4 ± 0.1	43.7 ± 0.7	59.1 ± 0.8

Table A.6: LAS objects by component (packed bytes; `bench_levels`). The LAS-2020/845 reference set shares the Simplified Dilithium-II dimensions. The challenge c is fixed; the response z grows with $n + \ell$ and dominates the signature at every setting. The target-setting column is the body table table 3.5.

Component	Simplified Dilithium-II $(4, 4)$	Simplified Dilithium-III $(6, 5)$	Simplified Dilithium-V $(8, 7)$
c (challenge)	32	48	64
z (response)	4608	6688	9120
Signature (c, z)	4640	6736	9184
z as % of signature	99.3%	99.3%	99.3%
Public key pk	2944	4416	5888
Secret key sk	512	704	960
Statement Y (LAS only)	2944	4416	5888
Witness y (LAS only)	512	704	960
Pre-signature (LAS only)	4640	6736	9184

```
}

int las_ext(las_sk *y, const las_sig *sig, const las_sig *presig,
            const las_pk *Y, const las_pp *pp) {
```

Table A.7: Source files of the implementation, classified by how the reference is used. The lattice core is reused unchanged; the only modified file is the `Makefile`; the scheme, serialization, application, and evaluation are new.

Category	Files	Role
Reused unchanged	`poly/polyvec/ntt/reduce/rounding`, `packing`, `sign`, `fips202`, `params`	the entire lattice core
Modified	`Makefile`	compile new targets
Added	`las.{c,h}`, `serialize.{c,h}`, `relation_zk.{c,h}` (+ LaZer bridge), `basesig`, `setup`, `relation`, tests, benchmarks	this work

```
  poly Ay[LAS_N];
  unsigned int j;
  for(j = 0; j < LAS_M; ++j) { /* s = z - z_hat */
    poly_sub(&y->s[j], &sig->z[j], &presig->z[j]);
    poly_reduce(&y->s[j]);
  }
  las_Amul(Ay, pp, y->s); /* check A*s == Y */
  for(j = 0; j < LAS_N; ++j)
    if(!poly_equal(&Ay[j], &Y->t[j]))
      return -1;
  return 0;
}
```

A.13 Settling on a real node

The transaction sizes of section 3.7 are arithmetic over the serialisation of [15, 13, 23]. This appendix records what happened when transactions of those shapes were put to a real Bitcoin client, in two stages that must not be conflated: the first carries lattice-sized objects past a *stock* node, the second verifies a lattice signature on a *patched* one. Full method, scope and caveats are in `docs/03-results/TWO_LEG_REAL_CLIENT_EXPERIMENT.md`; the evidence is under `evidence/btc_regtest/` and `evidence/btc_las_node/`.

Carriage on stock Bitcoin Core. Against an unmodified Bitcoin Core v31.1 regtest node (source commit `9be056a8a72b`), three transactions were constructed

with the client's own test framework, offered to two nodes differing only in relay policy, broadcast and mined.

transaction	vsize (vB)	weight (WU)	default relay policy
1-in/1-out P2WPKH, signed by the node	110	437	accepted
1-in/1-out P2TR key path	111	444	accepted
carriage of a LAS signature and key	2,939	11,753	`bad-witness-nonstandard`

Three observations. The two reference spends reproduced the published figures the size model self-checks against, so that model is now one a client has confirmed rather than one this study asserts. The carriage transaction is refused by default relay policy — the reason string is the node's own — yet is consensus-valid, was mined, and the default-policy node accepted the containing block: standardness and validity are distinct questions, and only the second bears on whether a post-quantum settlement is possible. Finally, because a signature of 6,736 bytes and a key of 4,416 bytes each exceed the 520-byte consensus limit on a witness stack item, they travel as 13 and 9 chunks respectively, which is why this transaction carries 25 witness items where an ordinary payment carries two.

Nothing here verifies a lattice signature. Stock Script has no such opcode; the spend is authorised by an ordinary Schnorr signature and the lattice bytes are discarded from the stack.

Verification on a patched node. [24] reserves the `OP_SUCCESS` opcodes as an upgrade path. Taking one of them (0xbb) and defining it as a lattice verification opcode over the byte interface of section 2.3, with the signed message being the transaction digest of [23], gives a node that can settle a spend carrying no elliptic-curve signature at all. Such a spend was accepted and mined (2,917 vB, 11,667 WU, 24 witness items); the consensus diff is recorded as a patch against the same release (`sha256:6fb8161d5b2b`) and applies to a pristine checkout of it.

That a node accepts a transaction proves little on its own, so each case was put to both the patched node and a stock one. On the stock node the opcode is still `OP_SUCCESS`, so it accepts every variant unconditionally — which is what makes it a control, since it cannot be reacting to the signature. 9 mutations were tried: altering the payment's value or destination, spending a different coin, computing the digest against a false input value, presenting a signature made for another

transaction or over something that is not a transaction digest at all, truncating or transposing a chunk, and substituting a key the output never committed to. Every one was refused by the patched node and accepted by the stock node, which attributes the refusals to the new rule rather than to some other defect.

Three limits travel with this. A patched node is not Bitcoin, so the claim that configurations 2 and 3 cannot settle on Bitcoin as deployed is unaffected. Implementing one of the routes sketched in section 3.7 demonstrates feasibility and takes no position on which route should be adopted. And the exercise is functional: no security argument is offered for the new rule, and it is not wired into the swap protocol.

What the rule charges a validating node. Changing a consensus rule buys something and charges something, and the charge falls on every node that ever validates the chain, so the cost was measured rather than left as a remark. The quantity that matters is script-validation time per input, since that is what scales with the number of signatures in a block. Timed over 10 repetitions × 200 iterations, paired and interleaved, one `OP_CHECKLASSIGVERIFY` costs 374±9 μs against 33.0±3.3 μs for a BIP340 Schnorr check and 31.7 μs for ECDSA — 11.3× and 11.8× respectively. Both sides start from *serialized* bytes and parse inside the timed call, as the interpreter does per input; timing a packed lattice verifier against an already-parsed curve key would charge the wire codec to one side only. The reject path is a *valid* signature checked against a different 32-byte digest, so every stage before the final challenge comparison must still run — a corrupted signature can trip an early structural check, which would measure how quickly the verifier gives up rather than what a rejection costs. Rejection comes to 372 μs, 1.00× an acceptance. The narrow reading is the only one supported: a LAS signature that fails at the final comparison costs a node approximately as much as one that verifies, so there is no early exit to be had on this path. It says nothing about the rule's denial-of-service surface more broadly — malformed inputs take other paths, which were not timed. Runner `scripts/run_btc_las_bench.sh`; evidence under `evidence/btc_las_bench/`.

Three caveats bound those figures, and they are separate claims. First, the curve baseline is a pinned `libsecp256k1-zkp` — a fork of `libsecp256k1` running the same verification algorithms, but not the library the patched client links, so these are not Bitcoin Core's own numbers. Second, the security of the consensus

modification remains unanalysed: a timing figure is not a safety argument, and soundness, denial-of-service surface and upgrade path are all untouched. Third, the two schemes are not at a matched security level — `secp256k1` offers roughly 128-bit *classical* security, whose engineering match is Simplified Dilithium-II, while the node compiles the rule at Simplified Dilithium-III because that is what it actually runs. The ratios therefore measure the curve against a deliberately stronger lattice setting and so *overstate* what the rule costs relative to a level-matched pairing; stating the direction of that bias is the point, and neither pairing is a security-equivalence claim. Finally, this is a per-input cryptographic cost only: block download, UTXO lookup, script parsing and signature caching are excluded, and the unmodified client pays those too.

Bibliography

[1] Lukas Aumayr, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Kristina Hostáková, Matteo Maffei, Pedro Moreno-Sanchez, and Siavash Riahi. Generalized channels from limited blockchain scripts and adaptor signatures. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 635–664. Springer, 2021.

[2] Tim Beiko. EIP-7773: Hardfork meta — glamsterdam. Ethereum Improvement Proposals, no. 7773, 2024. Records which proposals are scheduled for, or declined from, the Glamsterdam upgrade. `https://eips.ethereum.org/EIPS/eip-7773`.

[3] Bitcoin Core developers. Transaction relay and mining policy limits (`src/policy/policy.h`). Source code, 2026. `https://github.com/bitcoin/bitcoin/blob/master/src/policy/policy.h`; `MAX_STANDARD_TX_WEIGHT`, `MAX_STANDARD_P2WSH_STACK_ITEM_SIZE`.

[4] Blockstream Research. secp256k1-zkp: Ecdsa adaptor signature module. Software library, 2024. `https://github.com/BlockstreamResearch/secp256k1-zkp`, commit 95b9835.

[5] Maxime Buser, Rafael Dowsley, Muhammed Esgin, Clémentine Gritti, Shabnam Kasra Kermanshahi, Veronika Kuchta, Jason Legrow, Joseph Liu, Raphaël Phan, Amin Sakzad, et al. A survey on exotic signatures for post-quantum blockchain: Challenges and research directions. *ACM Computing Surveys*, 55(12):1–32, 2023.

[6] CRYSTALS team. CRYSTALS-Dilithium reference implementation. Software repository, 2023. `https://github.com/pq-crystals/dilithium`, commit 2374d22.

[7] Renaud Dubois and Simon Masson. EIP-8051: Precompile for ML-DSA signature verification. Ethereum Improvement Proposals, no. 8051, October 2025. Draft. `https://eips.ethereum.org/EIPS/eip-8051`.

[8] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR transactions on cryptographic hardware and embedded systems*, pages 238–268, 2018.

[9] Muhammed F Esgin, Oğuzhan Ersoy, and Zekeriya Erkin. Post-quantum adaptor signatures and payment channel networks. In *European Symposium on Research in Computer Security*, pages 378–397. Springer, 2020.

[10] Lloyd Fournier. One-time verifiably encrypted signatures a.k.a. adaptor signatures. Manuscript, 2019. `https://github.com/LLFourn/one-time-VES`.

[11] Brook Heisler, Jorge Aparicio, and contributors. Criterion.rs: statistics-driven micro-benchmarking for Rust. Software repository, 2025. `https://github.com/criterion-rs/criterion.rs`, version 0.8.2.

[12] Ruslan Kysil, István András Seres, Péter Kutas, and Nándor Kelecsényi. poqeth: Efficient, post-quantum signature verification on ethereum. In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, pages 327–343, 2025.

[13] Johnson Lau and Pieter Wuille. BIP 144: Segregated witness (peer services). Bitcoin Improvement Proposal, 2015. `https://github.com/bitcoin/bips/blob/master/bip-0144.mediawiki`; defines the marker/flag and witness serialisation format.

[14] Lazer-crypto project. LaZer: a library for lattice-based zero-knowledge proofs. Software library, 2025. `https://github.com/lazer-crypto/lazer`; accompanies the CCS 2024 paper "The LaZer Library: Lattice-Based Zero Knowledge and Succinct Proofs for Quantum-Safe Privacy".

[15] Eric Lombrozo, Johnson Lau, and Pieter Wuille. BIP 141: Segregated witness (consensus layer). Bitcoin Improvement Proposal, 2015. `https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki`; defines the witness field, transaction weight and virtual size.

[16] Vadim Lyubashevsky. Lattice signatures without trapdoors. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 738–755. Springer, 2012.

[17] Giulio Malavolta, Pedro Moreno-Sanchez, Clara Schneidewind, Aniket Kate, and Matteo Maffei. Anonymous multi-hop locks for blockchain scalability and interoperability. In *26th Annual Network and Distributed System Security Symposium, NDSS 2019*, 2019.

[18] National Institute of Standards, Technology (NIST), Thinh Dang, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, and Angela Robinson. Module-lattice-based digital signature standard, August 2024.

[19] Andrew Poelstra. Scriptless scripts. Presentation, Milan Bitcoin meetup / Blockstream, 2017. `https://github.com/apoelstra/scriptless-scripts`.

[20] Sebastien Riou, Jong-Yeon Park, Liga Anwar, Axel Poschmann, and Michael Hutter. Apples, oranges, and signatures: Pitfalls and methodology in ML-DSA benchmarking. Cryptology ePrint Archive, Paper 2026/1333, 2026. `https://eprint.iacr.org/2026/1333`.

[21] Eric Schorn. fips204: FIPS 204 ML-DSA in pure Rust. Software repository, 2025. `https://github.com/integritychain/fips204`, vendored at commit c948882.

[22] Peter W Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM review*, 41(2):303–332, 1999.

[23] Pieter Wuille, Jonas Nick, and Anthony Towns. BIP 341: Taproot: SegWit version 1 spending rules. Bitcoin Improvement Proposal, 2020. `https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki`.

[24] Pieter Wuille, Jonas Nick, and Anthony Towns. BIP 342: Validation of taproot scripts. Bitcoin Improvement Proposal, 2020. `https://github.com/bitcoin/bips/blob/master/bip-0342.mediawiki`.